scientific reports

Check for updates

OPEN

Collaborative filtering models an experimental and detailed comparative study

Devangam Bangaru Rajesh & Avadhesh Kumar✉

Recommender systems have become indispensable tools in various domains, such as e-commerce, entertainment, and social media, for delivering personalized user experiences. Collaborative Filtering (CF) is an essential technique in RS that leverages user similarity patterns to suggest items which align with individual preferences. This study presents an experimental comparative analysis of collaborative filtering-based recommender system methods including memory-based methods (KNN variants), model-based approaches (SVD, SVD++, co-clustering), and techniques based on neural networks (NCF, DeepFM, LightGCN). We conduct a thorough evaluation of these methods on the MovieLens benchmark datasets (100K, 1M, 25M) utilizing various metrics, such as RMSE, MAE, FCP, NDCG@10, Precision@10, Recall@10, and F1@10 Score, aiming to identify the most effective approaches and understand the advantages and disadvantages of each approach. Additionally, we provide detailed insights into the working mechanisms of each model. Our comprehensive analysis reveals the strengths and limitations of each method, offering critical insights for practitioners in selecting the most suitable recommender system technique based on specific requirements and constraints. The findings indicate that, on large datasets, neural and graph-based models achieve measurable improvements in both rating accuracy and top-k ranking tasks, with ranking gains observed upto 15%. Nonetheless, more straightforward approaches (KNN, SVD) continue to hold their ground in smaller datasets or low-resource environments because of their straightforward implementation and clarity in interpretation. The results highlight the importance of achieving a balance between computational expense, scalability, and model intricacy when choosing collaborative filtering techniques for practical implementations of recommender systems. We offer practical insights to assist professionals in selecting models that are suited to particular application needs and data attributes. This research enhances the understanding of collaborative filtering techniques and offers valuable insights for improving the performance of RS across diverse domains.

Keywords Collaborative filtering, Neural collaborative filtering, Recommender systems, Personalized recommendation, Similarity metric

The explosion of data with an increasing number of people using the internet daily leads to more chaos, and obtaining useful information from this large amount of data becomes a hefty task. Recommendation systems play an essential role in providing the user with the required information and, in turn, play a vital role in decision making[1–5].

To define formally, Recommender Systems(RS) are information filtering systems that aim to predict and recommend items (such as products, movies, music, and articles) to users based on their preferences, past interactions, and similarities to other users or items[6,7]. These systems are commonly used in e-commerce, content streaming platforms, social media, and various other online services to help users discover relevant and personalized items of interest[1,5,6,8–10].

There are generally three main types of RS (Fig. 1):

Content-based filtering (CBF)

This approach recommends items based on the similarity of their attributes or content to items that the user has liked or interacted within the past[6,11–13]. For example, if a user has previously enjoyed action movies in a movie recommendation system, the system might recommend other action movies based on their shared attributes[14–16].

School of Advanced Sciences, VIT-AP University, Inavolu, Amaravathi 522241, Andhra Pradhesh, India. ✉email: soni.iitkgp@gmail.com

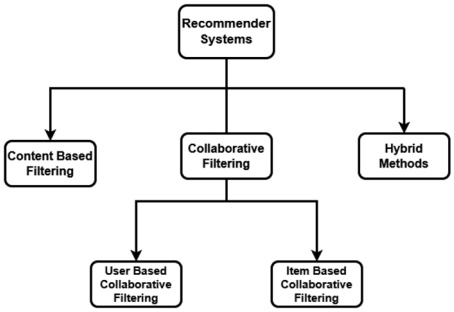


Fig. 1. Types of recommender systems.

Collaborative filtering (CF)

This approach recommends items based on the preferences of other users with similar tastes or behaviour[7,17]. Collaborative filtering can be further divided into two sub-types:

User-based collaborative filtering (UBCF)
Recommends items based on users' preferences similar to the target user. For instance, if two users have similar viewing histories on a video streaming platform, items watched by one user but not the other might be recommended to the second user[18].

Item-based collaborative filtering (IBCF)
Recommends items similar to the ones the user has liked or interacted with in the past[19]. It identifies similarities between items based on user interactions and recommends items similar to those the user has already expressed interest in the past.

Hybrid recommender systems

These systems combine multiple recommendation techniques to provide more accurate and diverse recommendations. By leveraging both CBF and CF approaches, hybrid recommender systems can mitigate the limitations of individual methods and offer more personalized recommendations to users[20].

We choose Collaborative Filtering based Recommender Systems(CF-RS) for its capacity to generate recommendations solely based on user interactions, sidestepping the need for item meta data and accommodating scenarios with sparse data or new items[18,21–23]. It fosters serendipity and diversity by suggesting items based on similar user preferences, tackling the cold start problem, and adapting dynamically to changing user tastes and item popularity. Scalable to large datasets, it efficiently handles millions of users and items independent of explicit user profiles or demographic data. By effectively incorporating implicit feedback like clicks and views, CF offers personalized recommendations across diverse domains, making it a versatile and powerful tool for enhancing user experiences in e-commerce, content streaming, and social networks[9,17,24].

The key highlights of this paper include the systematic and rigorous comparative evaluation of collaborative filtering methodologies encompassing both traditional and advanced techniques. This study focuses on memory based collaborative filtering methods, particularly K-Nearest Neighbourhood (KNN) variants, including KNNBasic, KNNBaseline, KNNWithMeans, and KNNZscore. We employ multiple similarity metrics, namely Cosine, Pearson Correlation Coefficient(PCC) and Mean Squared Difference(MSD), for analysis. We examine advanced model-based methods, specifically Singular Value Decomposition(SVD, SVD++) and co-clustering, emphasizing their effectiveness in improving recommendation accuracy.

This study also evaluates the advanced techniques such as Neural Collaborative Filtering(NCF), Deep Factorization Machines(DeepFM) and Light Graph convolutional Networks(LightGCN) to enhance the depth and contemporary relevance. These advanced models effectively capture complex, non-linear interactions between users and items, thereby offering robust solutions for issues related to sparsity and cold start scenarios.

The empirical analysis includes various datasets such as traditional benchmarks (Movielens 100k and 1m),and a large dataset MovieLens 25m. We conduct a thorough experimental analysis to identify and accurate trade-offs among accuracy, computational complexity, and scalability, providing insights for selecting optimal recommendation techniques suited to real-world application needs.

The article is structured as follows: Section 2 presents fundamental concepts and theoretical foundations pertinent to collaborative filtering methodologies. Section 3 offers comprehensive explanations of the employed procedures, encompassing both conventional and sophisticated CF approaches. Section 4 describes our experimental configuration, encompassing datasets assessment metrics, and computational factors. Section 5 provides a comprehensive analysis of experimental results of each method. Ultimately, Section 6 concludes the work and describes prospective directions for further research.

Preliminaries

Collaborative Filtering is a powerful technique used in RS to generate personalized recommendations by leveraging the preferences and behaviours of similar users. Unlike CBF, which relies on the attributes of items, CF focuses solely on user-item interactions. It assumes that users who have interacted with similar items will likely have identical preferences and interests in the future. This approach can be mainly divided into two main types: user-based collaborative filtering and item-based collaborative filtering (Fig. 2).

User-based collaborative filtering

In user-based collaborative filtering (UBCF) (Fig. 2a), recommendations are made by identifying users with tastes or preferences similar to the target user. The underlying assumption is that if two users have interacted with similar items in the past, they are likely to have similar tastes and preferences[7,17,22,23]. The process of generating recommendations involves the following steps:

User similarity calculation

It calculates the similarity between the target user and other users in the system based on their past interactions with items. Standard similarity metrics include Cosine similarity and Pearson correlation[25]. We will discuss this in more detail in the next section of this article.

Neighbourhood selection

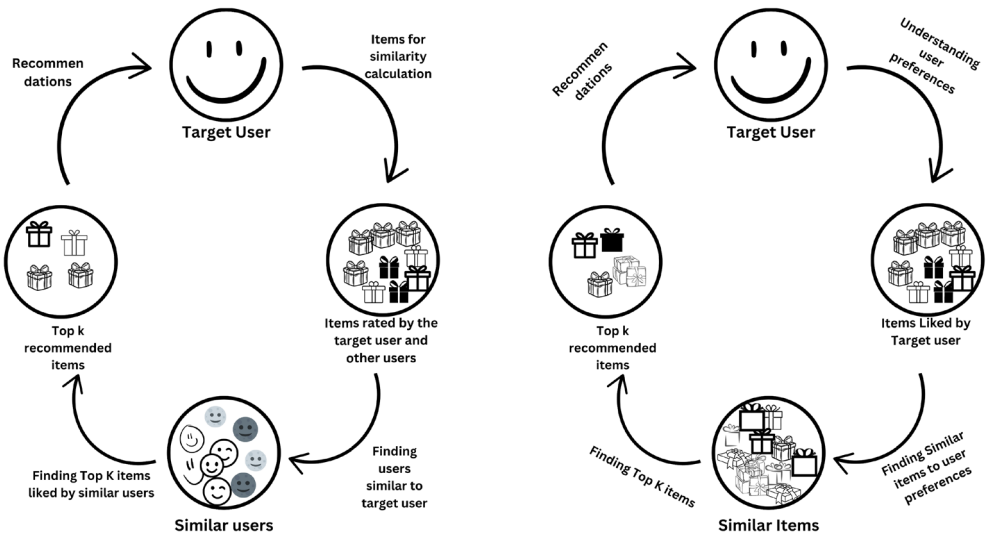
Identify a subset of users (neighbourhood) who are most similar to the target user based on the calculated similarity scores[26]. Here, we will use techniques to predict which top-k users are identical to the target user.

Prediction Generation: Predict the rating or preference of the target user for items they have not yet interacted with by aggregating the ratings of similar users[27]. The prediction function for UBCF is shown in equation 1. This equation calculates predictions by adjusting user ratings based on similarities with other users' deviations from their respective rating averages.

$$\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_{v \in N(i,u)} \text{sim}(u,v) \cdot (r_{v,i} - \bar{r}_v)}{\sum_{v \in N(i,u)} |\text{sim}(u,v)|} \tag{1}$$

Where:

$$\begin{aligned}
\hat{r}_{u,i} &: \text{Predicted rating of user } u \text{ for item } i \\
\bar{r}_u &: \text{Mean rating of user } u \\
\bar{r}_v &: \text{Mean rating of user } v \\
r_{v,i} &: \text{Rating of user } v \text{ for item } i \\
\text{sim}(u,v) &: \text{Similarity between users } u \text{ and } v \\
N(i,u) &: \text{neighbourhood of user } u \text{ for item } i
\end{aligned}$$



(a) User-Based Collaborative Filtering (b) Item-Based Collaborative Filtering

Fig. 2. Comparison of user-based and item-based collaborative filtering.

Item-based collaborative filtering

In Item-Based Collaborative Filtering (IBCF) (Fig. 2b), recommendations are made based on the similarity between items rather than users[27]. The assumption is that if two items have been interacted with by the same users, they are likely to be similar. The process of generating recommendations in item-based collaborative filtering is the same as that of UBCF, where we will calculate the similarity between items instead of users.

Prediction generation

Predict the rating or preference of the target user for items they have not yet interacted with based on their interactions with similar items. This prediction is typically generated by the prediction function given in equation 2. Unlike UBCF, item based prediction adjusts the target item's rating based on its similarity with items the user has previously interacted with, thereby improving the accuracy when items have richer rating data.

$$\hat{r}_{u,i} = \bar{r}_i + \frac{\sum_{j \in N(u,i)} \text{sim}(i,j) \cdot (r_{u,j} - \bar{r}_j)}{\sum_{j \in N(u,i)} |\text{sim}(i,j)|} \tag{2}$$

Where:

$$\begin{aligned}
\hat{r}_{u,i} &: \text{Predicted rating of user } u \text{ for item } i \\
\bar{r}_i &: \text{Mean rating of item } i \\
\bar{r}_j &: \text{Mean rating of item } j \\
r_{u,j} &: \text{Rating of user } u \text{ for item } j \\
\text{sim}(i,j) &: \text{Similarity between items } i \text{ and } j \\
N(u,i) &: \text{neighbourhood of items for user } u \\
&\quad \text{with respect to item } i
\end{aligned}$$

Collaborative filtering methods

Here, we comprehensively discuss the theoretical background of the selected collaborative filtering methods. We have also highlighted the advantages and disadvantages of each of the methods in Table 1.

We carefully choose the approaches for our comparison to encompass the range of trade-offs commonly encountered in real world recommender systems. Memory-Based techniques (KNN variants) offer simplicity,

Method	Advantages	Disadvantages	Core Feature
KNNBasic	Simple and easy to implement.	Computationally expensive for large datasets.	Non-parametric; suitable for smaller datasets.
	No explicit training phase required.	Sensitive to choice of similarity metric and value of k.	
	Robust to noisy data and outliers.		
KNNWithMeans	Reduces biases from different rating scales.	Sensitive to feature scaling and choice of similarity metric.	Mitigates biases through user/item rating adjustments.
	Improves prediction accuracy over basic KNN.		
KNNBaseline	Addresses data sparsity effectively.	Computationally more complex due to baseline calculations.	Uses baseline estimates to enhance predictions on sparse data.
	Improved accuracy in sparse environments.		
KNNZScore	Handles varied rating distributions effectively.	Assumes ratings follow a normal distribution.	Standardizes ratings to mitigate scale bias.
	Reduces bias from varying rating scales.		
SVD	Effectively manages sparsity through latent factors.	Computationally intensive training phase.	Factorizes the user-item matrix, capturing latent patterns.
	Enhances recommendation quality.		
SVD++	Incorporates implicit user feedback to enhance accuracy.	More computationally expensive compared to basic SVD.	Integrates explicit and implicit interaction signals.
	Better captures intricate user-item interactions.		
Co-Clustering	Efficient handling of large-scale datasets.	Sensitive to initialization and number of clusters chosen.	Discovers hidden structures via clustering of users/items.
	Effective in clustered and sparse data scenarios.		
NCF	Captures complex non-linear interactions.	Higher computational cost and complexity of hyperparameter tuning.	Integrates neural network layers with embedding methods.
	Strong performance with implicit feedback datasets.		
	Adaptable neural architecture for diverse datasets.		
DeepFM	Simultaneously models low-order and high-order interactions.	Requires extensive hyperparameter optimization; increased computational overhead.	Combines Factorization Machines (FM) with deep neural networks.
	Effective with large-scale, sparse datasets.		
LightGCN	Simplified graph convolutional structure; high accuracy with lower complexity.	Sensitive to graph structure and initial embedding dimensions.	Leverages graph-based convolution for embedding propagation.
	Scalable to large, sparse datasets with excellent ranking performance.		

Table 1. Comparison of collaborative filtering methods.

interpretability and ease of implementation. Model-Based approaches (SVD, SVD++, NMF, Co-clustering) enhance the management of sparsity and reveal the hidden latent patterns in the data, although at higher computational cost. In subsequent section we further introduce sophisticated neural-network based collaborative filtering methods to investigate non-linear and graph-structured interactions, yielding substantial accuracy enhancements for extensive datasets.

K-Nearest neighbour (KNN)

KNN is a fundamental supervised machine learning algorithm for classification and regression tasks. The intuition behind KNN is to predict the class of a data point by looking at the K nearest data points in the feature space. It was initially developed by Evelyn Fix and Joseph Hodges in 1951 and later expanded by Thomas Cover[28–30].

Input: User-item matrix R, target user u, number of neighbour k
Output: Predicted rating $\hat{r}_{u,i}$ for item i by user u
Let $N(u)$ be the set of k nearest neighbour of user u in the user-item matrix R
for each item i not rated by user u **do**
  Let $N(i,u)$ be the set of items rated by user u that are similar to item i
  if $|N(i,u)| < k$ **then**
    Skip item i as there are not enough similar items rated by user u
  else
    Compute the similarity between user u and each neighbour $v \in N(u)$
    Sort the neighbour in $N(u)$ based on their similarity to user u
    Select the top k most similar neighbour
    Compute the weighted average of their ratings for item i
    $\hat{r}_{u,i} \leftarrow$ Weighted average
  end if
end for

Algorithm 1. K-Nearest Neighbours (KNN).

The K-Nearest Neighbour (KNN) algorithm is a prevalent, instance-based collaborative filtering technique. It forecasts user ratings by averaging or weighing ratings from the most analogous users or products, ascertained by a similarity metric (e.g., Cosine, Pearson Correlation Coefficient, Mean Square Difference)[31,32]. The principal benefits of KNN encompass simplicity, convenience of implementation, the absence of a formal training phase, and resilience to noise[33]. KNN is hindered by high computational costs during prediction, sensitivity to the selection of similarity metrics and the parameter K, as well as scalability challenges with large datasets[32,34–36]. Below, we explore four notable variations of KNN.

KNNBasic
The basic idea is to assign the average of the closest neighbour to the query point. KNNBasic predicts ratings by averaging those from the k nearest neighbour(eq: (3)). Its simplicity makes it suitable for small datasets, though performance declines sharply with dataset size. Equation (3) represents the KNNBasic algorithm, where the predicted rating $\hat{r}_{u,i}$ for user u and item i.

$$\hat{r}_{u,i} = \frac{1}{k} \sum_{v \in N(i,u)} r_{v,i} \tag{3}$$

KNN Basic is ideal for small datasets or where interpretability and simplicity are prioritized. However, its effectiveness deteriorates rapidly with dataset scale.

KNNWithMeans
This variation adjusts each neighbour's rating by subtracting the mean rating of the respective user or item, aiming to mitigate biases or tendencies present in the data. KNNwithMeans enhances prediction accuracy and robustness by incorporating mean adjustments, especially in datasets with heterogeneous rating distributions. Equation (4) represents the KNNwithMeans algorithm.

$$\hat{r}_{u,i} = \bar{r}_u + \frac{1}{k} \sum_{v \in N(i,u)} (r_{v,i} - \bar{r}_v) \tag{4}$$

KNNBaseline

This extension addresses the challenges of data sparsity and incompleteness by adjusting predictions based on baseline estimates derived from the overall rating tendencies of users or items. These baseline estimates, represented by $b_{u,i}$, account for the inherent biases of both users and items. For instance, some users may consistently give higher or lower ratings than others, and certain items may generally receive better or worse ratings.

The baseline estimate $b_{u,i}$ for user u and item i is initialized as:

$$b_{u,i} = \mu + b_u + b_i \tag{5}$$

where μ is the global mean of all ratings, b_u is the user bias, and b_i is the item bias. The global mean μ is calculated as the average of all ratings in the dataset. These biases are iteratively updated during the model training phase to minimize the overall prediction error. This adjustment ensures that systematic biases are accounted for, leading to more accurate predictions, especially in cases with sparse data or when a user or item exhibits unusual rating behavior. The prediction process is represented by equation (6):

$$\hat{r}_{u,i} = b_{u,i} + \frac{1}{k} \sum_{v \in N(i,u)} (r_{v,i} - b_{v,i}) \tag{6}$$

In this equation, $N(i,u)$ represents the set of k-nearest neighbours of user u who have rated item i.

KNNZScore

KNNZScore, or Standardization, involves scaling the dataset's features to have a mean of zero and a standard deviation of one (i.e., transforming the data to follow a standard normal distribution). This transformation ensures that all features contribute equally to the distance calculations, preventing larger-scale features from dominating the distance metric. This variation leads to more robust predictions and improved performance, particularly in datasets with diverse rating distributions. Equation (7) is used to calculate the Z-score $z_{u,i}$ for user u and item i. $\bar{r}_i$ represents the mean rating of item i, calculated as the average of all ratings given to item i. Similarly, σ_i denotes the standard deviation of ratings for item i. A higher of σ_i indicates more variability in ratings for item i, while a lower suggests that ratings are more tightly clustered around the mean. The standardized ratings are then utilized for predictions in the KNNZScore

$$z_{u,i} = \frac{r_{u,i} - \bar{r}_i}{\sigma_i} \tag{7}$$

Having a comprehensive discussion of all KNN methods, let's discuss the similarity metrics that determine the KNN's performance.

Similarity metrics

Similarity metrics are the measures used to quantify the similarity between the given users or items. Here, we discuss the three similarity metrics we use in experimental analysis. For a better understanding, the equations of similarity function for UBCF and IBCF are written separately even though they are very similar.

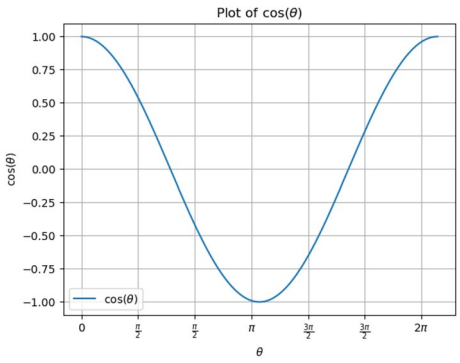


Fig. 3. Cosine graph.

Cosine similarity

This is the most popular and widely used similarity metric. It calculates the cosine of the angle between the two vectors(user or item), which gives the directional similarity neglecting their magnitude[37,38]. Equation (8) provides a value between -1 and 1, where 1 indicates perfect similarity, -1 indicates perfect dissimilarity, and 0 represents orthogonality (i.e., the vectors are perpendicular to each other) (Fig. 3).

In UBCF, cosine similarity is used to measure the similarity between two users based on their rating patterns[38]. The cosine similarity $\cos_\mathrm{sim}(u, v)$ between two users u and v is calculated using the following formula:

$$\cos_\mathrm{sim}(u, v) = \frac{\mathrm{r}_u \cdot \mathrm{r}_v}{\|\mathrm{r}_u\| \|\mathrm{r}_v\|} \tag{8}$$

In Equation (8), r_u and r_v represent the rating vectors of users u and v, respectively. The dot product in the numerator calculates the similarity between the rating vectors, while the denominators normalize the similarity score by the magnitudes of the vectors[13,38].

In IBCF, cosine similarity is utilized to measure the similarity between two items based on the ratings given by users. The cosine similarity $\cos_\mathrm{sim}(i, j)$ between two items i and j is calculated as follows:

$$\cos_\mathrm{sim}(i, j) = \frac{\mathrm{r}_i \cdot \mathrm{r}_j}{\|\mathrm{r}_i\| \|\mathrm{r}_j\|} \tag{9}$$

In Equation (9), r_i and r_j denote the rating vectors of items i and j, respectively.

Cosine similarity is beneficial when dealing with high-dimensional data or when the magnitude of the vectors is not relevant to the similarity measurement[13,39]. Computing cosine similarity involves simple arithmetic operations such as dot products and vector norms, making it computationally efficient, especially for large datasets and real-time applications.

Despite its simplicity and computational efficiency, it has few drawbacks. Cosine similarity does not capture non-linear relationships between vectors. It assumes that the angle between vectors adequately represents their similarity, which may not always be the case in complex data distributions. It can be sensitive to noise in the data, mainly when dealing with high-dimensional spaces or datasets with low signal-to-noise ratios. Noise can distort the angle between vectors, leading to inaccurate similarity assessments[13,38,39]. While cosine similarity is scale-invariant, it may exhibit bias towards more significant magnitude vectors, especially when the angle between vectors is small. This bias can affect the perceived similarity, particularly in datasets with imbalanced distributions of magnitudes[13].

Pearson correlation coefficient

The Pearson Correlation Coefficient (PCC)[40] is a statistical measure that quantifies the linear correlation between two data sets. The PCC is calculated as the covariance of two variables divided by the product of their standard deviations. It provides a normalized measurement of the covariance, ensuring that the result always falls between -1 and 1, where a positive indicates a positive correlation, a negative suggests a negative correlation, and 0 means no correlation[13].

In UBCF, the formula for computing the Pearson Correlation Coefficient $\mathrm{PCC}(u, v)$ between two users u and v is given by:

$$\mathrm{PCC}(u, v) = \frac{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)(r_{v,i} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_{i \in I_{uv}} (r_{v,i} - \bar{r}_v)^2}} \tag{10}$$

In Equation (10), I_{uv} represents the set of items rated by both users u and v. $r_{u,i}$ and $r_{v,i}$ denote the ratings of users u and v for item i, respectively. $\bar{r}_u$ and $\bar{r}_v$ are the mean ratings of users u and v, respectively.

In IBCF, PCC is used to quantify the similarity between two items based on users' rating patterns. The $\mathrm{PCC}(i, j)$ between two items i and j is calculated using the following formula:

$$\mathrm{PCC}(i, j) = \frac{\sum_{u \in U_{ij}} (r_{u,i} - \bar{r}_i)(r_{u,j} - \bar{r}_j)}{\sqrt{\sum_{u \in U_{ij}} (r_{u,i} - \bar{r}_i)^2} \sqrt{\sum_{u \in U_{ij}} (r_{u,j} - \bar{r}_j)^2}} \tag{11}$$

In Equation (11), U_{ij} represents the set of users who have rated both items i and j. $r_{u,i}$ and $r_{u,j}$ denote the ratings of user u for items i and j, respectively. $\bar{r}_i$ and $\bar{r}_j$ are the mean ratings of items i and j, respectively.

PCC remains invariant under separate changes in location and scale for the two variables. Geometrically, PCC is similar to Cosine Similarity, where PCC is mean-centric, whereas cosine is zero-centric. Since PCC relies on historical ratings to calculate similarity, it may struggle to provide accurate recommendations for new entities[13].

Mean square difference

The fundamental idea behind MSD similarity is to quantify the similarity between two entities (either users or items). The intuition is that if two users (or items) have provided similar ratings across a significant number of items (or have been rated similarly by many users), they are likely to be identical in preferences or characteristics.

In UBCF, MSD measures the dissimilarity between two users based on their ratings across commonly rated items is given by equation 12, where u and v are users, I_{uv} is the set of items both users have rated, $r_{u,i}$ and $r_{v,i}$ are the ratings given by u and v to item i, and $|I_{uv}|$ represents the number of items both users have rated.

$$\text{MSD}_{\text{user}}(u,v) = \frac{1}{|I_{uv}|} \sum_{i \in I_{uv}} (r_{u,i} - r_{v,i})^2 \tag{12}$$

In IBCF, MSD quantifies the dissimilarity between two items based on the ratings received from all users who have rated both. The formula is given in the equation 13, where i and j are items, U_{ij} is the set of users who have rated both items, $r_{u,i}$ and $r_{u,j}$ are the ratings given by u to items i and j, and $|U_{ij}|$ represents the number of users who have rated both items.

$$\text{MSD}_{\text{item}}(i,j) = \frac{1}{|U_{ij}|} \sum_{u \in U_{ij}} (r_{u,i} - r_{u,j})^2 \tag{13}$$

Building upon our discussion of the MSD as a measure of dissimilarity in CF, We use inverse transformation to get the similarity score.

$$\text{Similarity}_{\text{user}}(u,v) = \frac{1}{1 + \text{MSD}_{\text{user}}(u,v)} \tag{14}$$

Similarly, in IBCF, the similarity between two items can also be calculated from MSD; using the same transformations as for the user-based case, the similarity can be expressed as:

$$\text{Similarity}_{\text{item}}(i,j) = \frac{1}{1 + \text{MSD}_{\text{item}}(i,j)} \tag{15}$$

Inverse transformation adapts well to varying ranges of MSD. It ensures that the similarity scores are always positive and normalized, which is crucial for stable and predictable performance in recommendation systems. MSD is sensitive to variations in ratings, effectively capturing even slight differences. This sensitivity allows the model to differentiate clearly between entities (users or items) based on their rating patterns, which can enhance the quality of recommendations. The effectiveness of MSD can depend on the scale of rating data. If the rating range is extensive, the squared differences can become disproportionately large, potentially skewing the similarity results. Two pairs could have the same MSD but vastly different numbers of ratings in common, which might misrepresent their actual level of similarity or dissimilarity.

The selection of Cosine Similarity, PCC, and MSD as similarity metrics for KNN-based methods facilitates a thorough performance comparison, given their distinct characteristics. Cosine Similarity emphasizes the orientation of rating vectors, rendering it suitable for users employing varying rating scales, while also being computationally efficient. PCC normalizes ratings by deducting the user's mean, thereby accurately reflecting authentic similarities in rating patterns and demonstrating efficacy with dense data. MSD imposes penalties for substantial discrepancies in ratings, offering a direct assessment of disagreement while effectively addressing absolute rating differences. Comparing these metrics facilitates a comprehensive assessment, allowing for the identification of the most appropriate metric for particular use cases, enhancing performance, and ensuring the reliability of the recommendation system. Features of these similarity metrics are also presented in tabular form in Table 2.

Model-based methods

Singular value decomposition (SVD) and SVD++

Singular Value Decomposition (SVD)[41] is a matrix factorization technique widely used in CF to decompose the user-item rating matrix R into three matrices: U (user latent factors), Σ (singular s), and V^T (item latent factors)[42]. The approximation of matrix R using SVD is given by:

$$R \approx U \Sigma V^T \tag{16}$$

This factorization facilitates the prediction of missing ratings, thus enhancing recommendation systems. SVD decreases the dimension of the rating matrix by extracting its latent factors[43]. With SVD, we turn the recommendation problem into an optimization problem that deals with how good we are at predicting the ratings for items given to a user.

Similarity Metric	Advantages	Disadvantages	Core Feature
Cosine Similarity	Effective with varied rating scales.	Ignores user preferences like leniency or harshness.	Focuses on direction of ratings, not magnitude.
	Computationally efficient.		
PCC	Captures genuine rating similarities.	Performs poorly in sparse datasets.	Accounts for rating bias by normalizing data.
	Performs well in dense datasets.		
MSD	Directly measures disagreement in ratings.	Over-penalizes minor rating differences.	Focuses on minimizing absolute rating differences.
	Handles large rating differences well.		

Table 2. Comparison of similarity metrics.

SVD++ extends SVD by incorporating implicit feedback, providing a more nuanced approach to understanding user preferences[34]. The predictive model in SVD++ is represented as:

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^T \left(p_u + |N(u)|^{-\frac{1}{2}} \sum_{j \in N(u)} y_j \right) \tag{17}$$

where:

$$\begin{aligned}
\hat{r}_{ui} &: \text{predicted rating by user } u \text{ for item } i,\\
\mu &: \text{global average rating},\\
b_u &: \text{user } u\text{'s bias}\\
b_i &: \text{item } i\text{'s bias},\\
q_i &: \text{latent factor vector for item } i,\\
p_u &: \text{latent factor vector for user } u,\\
y_j &: \text{latent factors for items interacted with by } u \text{ implicitly},\\
N(u) &: \text{set of items interacted with by } u \text{ implicitly}.
\end{aligned}$$

Input: User-item rating matrix R, number of singular s k
Output: Predictions for all missing entries in R
Perform singular decomposition on R to get matrices U, Σ, V^T
U is an $m \times m$ orthogonal matrix where m is the number of users
Σ is an $m \times n$ diagonal matrix of singular s, n is the number of items
V^T is an $n \times n$ orthogonal matrix
if $k > \min(m, n)$ **then**
  Set $k = \min(m, n)$ {Adjust k to the maximum possible rank}
end if
Reduce U to U_k by taking the first k columns
Reduce Σ to Σ_k by keeping only the top $k \times k$ block
Reduce V^T to V_k^T by taking the first k rows
Compute $R_k = U_k \Sigma_k V_k^T$
for each user u **do**
  for each item i that user u has not rated **do**
    Predict rating $\hat{r}_{u,i}$ using $(U_k \Sigma_k V_k^T)_{u,i}$
    Store $\hat{r}_{u,i}$ in the predicted ratings matrix
  end for
end for
Return the matrix of predicted ratings

Algorithm 2. SVD for Collaborative Filtering.

Input: User-item rating matrix R, implicit feedback matrix F, number of factors k, learning rate η, regularization term λ
Output: Updated matrices U, P, Y and Q for predictions
Initialize user factors matrix U, item factors matrix Q
Initialize user implicit feedback factors matrix Y
Initialize user biases b_u and item biases b_i
for each epoch **do**
  for each (u, i) in R with known rating r_{ui} **do**
    Compute implicit feedback factor:

$$y_u = \sum_{j \in I_u} \frac{y_j}{\sqrt{|I_u|}}$$

    Compute the prediction error $\hat{e}_{ui} = r_{ui} - \mu - b_u - b_i - (q_i^T(p_u + y_u))$
    Update biases:

$$b_u \leftarrow b_u + \eta(\hat{e}_{ui} - \lambda b_u)$$
$$b_i \leftarrow b_i + \eta(\hat{e}_{ui} - \lambda b_i)$$

    Update factors:

$$p_u \leftarrow p_u + \eta(\hat{e}_{ui} q_i - \lambda p_u)$$
$$q_i \leftarrow q_i + \eta(\hat{e}_{ui}(p_u + y_u) - \lambda q_i)$$

    for each j in I_u **do**
      Update implicit factors:

$$y_j \leftarrow y_j + \eta \left(\frac{\hat{e}_{ui}}{\sqrt{|I_u|}} q_i - \lambda y_j \right)$$

    end for
  end for
  Optionally reduce η to ensure convergence
end for
Return U, P, Y, Q

Algorithm 3. SVD++ for Collaborative Filtering.

In the provided SVD++ algorithm, the user-item rating matrix *R* stores explicit ratings, while the implicit feedback matrix *F* captures non-explicit user interactions such as views or clicks[44]. The algorithm utilizes latent factor matrices *U*, *Q*, and *Y* for users, items, and implicit feedback, each represented by *k* factors to capture underlying data patterns. The learning rate η and regularization term λ are crucial hyper-parameters; η controls the update step size during gradient descent, whereas λ mitigates overfitting by penalizing significant parameters. User and item biases, b_u, and b_i adjust for individual tendencies beyond interaction effects. The iterative process over epochs allows the model to refine its predictions, minimizing the prediction error $\hat{e}_{ui}$, which is the discrepancy between observed and predicted ratings. The updates to the biases and factor matrices during each epoch are driven by this error, ensuring the model progressively learns to approximate the absolute ratings more accurately while also considering both explicit and implicit user feedback.

Both SVD and SVD++ provide high accuracy in predicting unknown ratings, enhancing the effectiveness of RS. SVD effectively manages sparse datasets, which are common in CF applications. SVD handles large datasets efficiently, making it suitable for real-world recommender systems. SVD can address the cold start problem to some extent by leveraging user and item factors to make predictions for new users or items with limited interaction history. SVD++ includes explicit and implicit feedback, capturing a broader spectrum of user preferences[45]. Computational intensity increases with larger datasets, which can be challenging for real-time applications. SVD++ introduces additional complexity due to managing more parameters and capturing implicit interactions, requiring more computational resources and time. Struggles with new users and items that lack sufficient historical data to derive meaningful latent factors[45].

Co-clustering

Co-clustering, or bi-clustering, is a technique that simultaneously partitions data objects (rows) and data features (columns) into coherent subgroups. These subgroups, called co-clusters, represent subsets of objects and features that exhibit similar behavior or share underlying patterns[46]. In CF, this means clustering both users and items concurrently to uncover latent patterns in user-item interactions. Co-clustering in collaborative filtering addresses challenges such as data sparsity and the cold start problem[35,47]. It enhances recommendation accuracy by clustering users and items based on similar rating behaviors, efficiently handles large datasets, and provides deeper insights into user-item interactions. The mathematical foundation of co-clustering in CF is based on optimizing a cost function that reflects the quality of clustering:

$$\min_{U,I} \sum_{u,i} \left(r_{ui} - \mu - u^T U i - b_u - b_i\right)^2 + \lambda(||U||_F^2 + ||I||_F^2) \tag{18}$$

where R is the user-item rating matrix, r_{ui} is the rating of user u for item i, μ is the global mean rating, b_u and b_i are user and item biases, U and I are user and item cluster matrices, and λ is the regularization parameter.

Input: User-item rating matrix R, number of user clusters k_u, number of item clusters k_i, regularization parameter λ
Output: Optimized user and item cluster matrices U and I
Initialize user cluster matrix U and item cluster matrix I randomly
Initialize convergence flag as False
while not convergence **do**
  Step 1: Update User Clusters
  for each user cluster u **do**
    Compute optimal user cluster centroids minimizing $\|R - UI^T\|_F^2 + \lambda(\|U\|_F^2 + \|I\|_F^2)$
  end for
  Step 2: Update Item Clusters
  for each item cluster i **do**
    Compute optimal item cluster centroids minimizing $\|R - UI^T\|_F^2 + \lambda(\|U\|_F^2 + \|I\|_F^2)$
  end for
  Check for convergence: if changes in U and I are below a threshold, set convergence flag to True
end while
Return U and I

Algorithm 4. Co-clustering for Collaborative Filtering.

Co-clustering in CF offers several advantages, including improved scalability by reducing the dimensionality of the user-item matrix, which makes it more manageable for large datasets[48]. This method effectively addresses the sparsity problem in user-item interactions by clustering similar users and items, thus enhancing recommendation accuracy. Furthermore, co-clustering can reveal latent relationships between users and items, enriching the analysis and potentially improving recommendation diversity. However, it also has disadvantages. The initial choice of the number of clusters can greatly impact performance, and poor initial decisions can lead to sub-optimal clustering that doesn't adequately capture the underlying structure of the data. Additionally, co-clustering assumes that the interactions within each cluster are similar. However, this might not always be accurate, leading to overly simplistic models that could miss essential but subtle differences in preferences among users or items.

Although conventional Collaborative filtering methods proficiently tackle numerous practical recommendation situations, they possess intrinsic limitations, especially regarding non-linear interaction modelling and scalability with extensive, sparse datasets. Recent advancements address those constraints by utilizing neural and graph-based methodologies, including Neural Collaborative Filtering (NCF), Deep Factorization Machines(DeepFm), and graph-based techniques(e.g., LightGCN), which more effectively capture

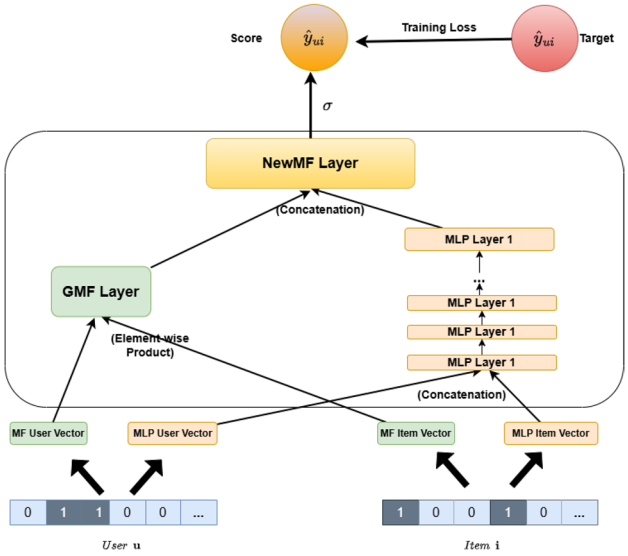


Fig. 4. Neural collaborative filtering.

intricate user-item interaction dynamics. These sophisticated strategies will be examined in detail in the next section.

Neural network based collaborative filtering methods

Neural network based algorithms have emerged to effectively represent complex, non-linear user-item interactions and handle scaling difficulties in big, sparse datasets, overcoming the constraints of previous collaborative filtering methods. Here we analyze three prominent neural network driven collaborative filtering techniques: Neural Collaborative Filtering (NCF), Deep Factorization Machines (DeepFM), and Light Graph Convolutional Networks (LightGCN)

Neural collaborative filtering

Neural Collaborative Filtering has been developed as a novel paradigm in recommender systems, overcoming limitations of conventional collaborative filtering methods[49]. Traditional collaborative filtering approaches generally depend on linear assumptions on user-item interactions, contrasting their ability to represent intricate, non-linear relationships frequently found in real-world data. NCF, developed by He et al. (2017)[49], address this limitation using deep neural networks, offering a sophisticated framework that explicitly models both linear and complex non-linear interactions.

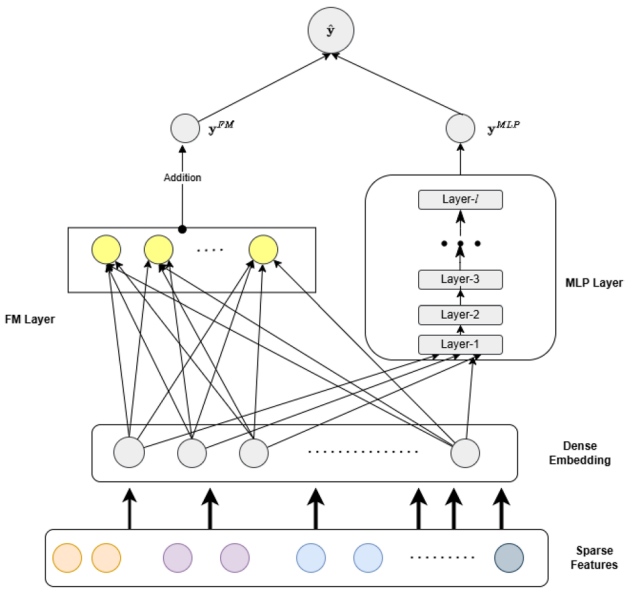


Fig. 5. Deep factorization machines.

NCF includes two independent components: Generalized Matrix Factorization(GMF) and Multi-layer perceptron (MLP)). GMF effectively captures linear interactions through the inner products of user-item embedding, but the MLP component excels at modelling intricate, non-linear relationships through numerous hidden neural layers. This hybrid architecture enables NCF to concurrently learn both linear and deep non-linear patterns from interaction data, improving recommendation accuracy, especially for implicit feedback datasets.

The NCF architecture consists of three main layers as given in Fig. 4.

- **Embedding Layer:** It converts the sparse user and item interactions into dense embedding vectors learned during training, representing users and items in a continuous latent space.
- **Hidden Layer (MLP Component):** User and item embeddings are concatenated and passed through multiple hidden layers, typically employing Rectified Linear Unit(ReLU) activations. This allows the neural network to effectively capture non-linear interactions.
- **Prediction Layer:** Outputs from GMF and MLP components are concatenated and fed into a final sigmoid layer to estimate the likelihood of a user-item interaction.

Mathematical formulation Let $\mathrm{p}_u \in \mathbb{R}^k$ and $\mathrm{q}_i \in \mathbb{R}^k$ denote embeddings for user u and item i, respectively. The Generalized Matrix Factorization (GMF) component models user-item interactions via an element-wise product:

$$y_{ui}^{GMF} = \mathrm{p}_u \odot \mathrm{q}_i \tag{19}$$

The Multi-Layer Perceptron (MLP) component captures complex, higher-order interactions by processing concatenated user and item embeddings through multiple nonlinear layers:

$$y_{ui}^{MLP} = \mathrm{MLP}([\mathrm{p}_u, \mathrm{q}_i]) \tag{20}$$

The final prediction is derived by concatenating these GMF and MLP outputs and feeding them through a sigmoid activation layer:

$$\hat{y}_{ui} = \sigma\left(\mathrm{h}^T[y_{ui}^{GMF}, y_{ui}^{MLP}]\right) \tag{21}$$

where $\sigma(\cdot)$ represents the sigmoid activation function, and h denotes the learnable parameters of the final prediction layer.

Training and Optimization:

Neural Collaborative Filtering models typically employ implicit feedback data and are optimized using binary cross-entropy loss, formally expressed as:

$$\mathcal{L} = -\sum_{(u,i)\in\mathcal{D}} (y_{ui}\log(\hat{y}_{ui}) + (1-y_{ui})\log(1-\hat{y}_{ui})) \tag{22}$$

In this formulation, $\mathcal{D}$ is the set of observed user-item interactions, y_{ui} indicates the actual binary interaction, and $\hat{y}_{ui}$ represents the predicted probability of interaction.

Computational Complexity: The per-interaction complexity of Neural Collaborative Filtering (NCF) primarily depends on embedding size (k), hidden-layer dimensions (h), and the number of hidden layers (L), and is summarized as:

$$O(kh + h^2L)$$

During training, the total complexity per epoch, with $|\mathcal{D}|$ interactions, is:

$$O(|\mathcal{D}|(kh + h^2L))$$

Advantages:

- Effectively models non-linear and linear relationships simultaneously.
- Enhanced performance on implicit feedback datasets.
- Versatile and adjustable neural architecture appropriate for various fields.

Limitations:

- Higher computational complexity compared to traditional methods
- Performance is sensitive to hyper-parameter tuning and model architecture

Deep factorization machines

Deep Factorization Machines (DeepFM) are introduced by Guo et.al(2017)[50]. It provides a powerful integrated approach for recommendation tasks, effectively capturing the lower order and higher order features in unified neural network architecture. Linear Regression and basic factorization models perform well in capturing linear and pairwise interactions but struggle with complex, non linear relationships among user-item features.

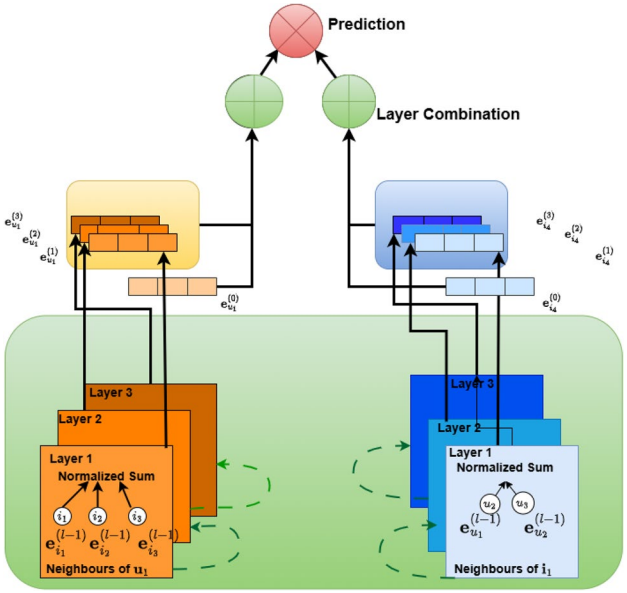


Fig. 6. Light graph convolution (LGC)[51].

DeepFM combines the strengths of Factorization Machines and Deep Neural Networks (DNNs) to address these limitations.

DeepFM integrates two separate functionalities: Factorization Machine component that explicitly models lower order features and DNN component proficient in capturing higher-order, non linear interactions. DeepFM uses shared embedding vectors, markedly decreasing model complexity and improving learning efficiency. This duel modelling strategy allows DeepFM to identify intricate feature interaction patterns, crucial for enhancing prediction accuracy.

The model architecture of DeepFM comprises of three primary layers (Fig. 5).

Embedding Layer Transforms raw input features into dense embeddings utilized by both FM and DNN components enhancing the efficiency and minimising the parameter redundancy.

Factorization Machines component captures the linear and pairwise interactions explicitly using embedding vectors, ensuring effective modelling of low-order feature interactions.

Deep Neural Network component process the embeddings through multiple fully connected layers with non linear activation functions (often ReLu), thereby capturing the higher order feature interactions.

Concatenation and Prediction Layer The outputs from the FM and DNN components are concatenated and input into a final sigmoid layer for the calculation of interaction probability.

Mathematical Formulation

The input for DeepFM is a feature vector $mathbf{x}$, where each feature is represented as embedding vectors $\mathrm{v}_i \in \mathbb{R}^k$. The FM component of the model interacts as:

$$y_{FM}(\mathrm{x}) = w_0 + \sum_{i=1}^{n} w_i x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} \langle \mathrm{v}_i, \mathrm{v}_j \rangle x_i x_j \tag{23}$$

Here, w_0 represents the global bias, w_i denotes the weight of feature i, and the dot product between embedding vectors v_i and v_j denoted by $\langle \mathrm{v}_i, \mathrm{v}_j \rangle$.

The DNN component of the model is defined as:

$$y_{DNN}(\mathrm{x}) = \sigma(W^{(L)}(\sigma(...\sigma(W^{(1)}\mathrm{x} + b^{(1)}))...) + b^{(L)}) \tag{24}$$

where $W^{(l)}$ and $b^{(l)}$ are weights and biases of the lth layer, respectively, and σ represents a non-linear activation function (e.g., ReLU).

The final prediction of the model combines both FM and DNN

$$\hat{y}(\mathrm{x}) = \sigma(y_{FM}(\mathrm{x}) + y_{DNN}(\mathrm{x})) \tag{25}$$

where the sigmoid function $\sigma(\cdot)$ converts the combined score into a predicted interaction probability.

Training and optimization DeepFM models generally employ binary cross-entropy loss for binary classification problems, which is defined as:

$$\mathcal{L} = -\sum_{(\mathrm{x},y)\in\mathcal{D}} [y\log(\hat{y}(\mathrm{x})) + (1-y)\log(1-\hat{y}(\mathrm{x}))] \tag{26}$$

Here, $\mathcal{D}$ represents the training dataset, y signifies the actual binary label, and $\hat{y}(\mathrm{x})$ symbolizes the anticipated probability of interaction as calculated by equation (26).

Computational Complexity: The computational complexity of the DeepFM per prediction primarily depends on embedding size k, the average number of non-zero input features n, and the dimensions of the DNN hidden layers h. It can be summarized as:

$$O(kn^2 + kh + h^2L) \tag{27}$$

During training, for a dataset of size $|\mathcal{D}|$, the complexity per epoch scales linearly:

$$O(|\mathcal{D}|(kn^2 + kh + h^2L)) \tag{28}$$

Advantages:

- Effectively captures lower order and higher order interactions simultaneously
- Utilizes shared embeddings, markedly decreasing the parameter count and improving training efficiency.
- Strong performance in sparse and high-dimensional contexts prevalent in real-world applications.

Limitations:

- Demands careful hyper-parameter tuning, including the embedding sizes and DNN configurations.
- Increased computational complexity relative to traditional linear or simple models.

DeepFM models performs well in recommendation and prediction tasks involving high dimensional and sparse data. It is particularly effective in capturing complex interactions between user and item features where traditional linear or basic FM models fall short. Moreover DeepFM reuses embedding, optimising parameters effectively and achieving a strong balance between predictive accuracy and computational efficiency.

LightGCN

LightGCN, introduced by He et al. (2020)[51], is a streamlined and efficient Graph Neural Network (GNN) approach explicitly tailored for collaborative filtering applications. In contrast to conventional GNNs, LightGCN eliminates superfluous complications, including non-linear activation functions and feature transformations, concentrating exclusively on neighbourhood aggregation within the user-item bipartite interaction graph. The optimised approach markedly improves recommendation precision and scalability, especially advantageous in sparse user-item interaction contexts prevalent in recommendation systems.

The model architecture of the LightGCN consists of four interconnected components:

Embedding Layer The embedding layer allocates learnable low-dimensional vector representations to every user and item. The embeddings are represented as $\mathrm{e}_u^{(0)} \in \mathbb{R}^k$ for user u and $\mathrm{e}i(0) \in \mathbb{R}^k$ for item i, serve as the foundational input to the model. Embeddings are initially randomized and iteratively optimized during the training process to accurately capture latent characteristics of users and items.

Graph Convolution Layer Graph Convolution Layer (Fig. 6) provide the foundation of LightGCN, facilitating the propagation of embeddings along the user-item interaction network. At each propagation layer, embeddings are updated by aggregating information from directly linked neighbours, with users aggregating embeddings from interacted items and vice versa. In contrast to the conventional GNNs, LightGCN employs linear neighbourhood aggregation, without applying non-linear activation functions or complex transformations, hence diminishing computational burden while preserving successful representation learning[51]. The propagation rule at each layer is mathematically expressed as:

$$\mathrm{e}_u^{(l+1)} = \sum_{i\in\mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u||\mathcal{N}_i|}}\mathrm{e}_i^{(l)}, \quad \mathrm{e}_i^{(l+1)} = \sum_{u\in\mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_i||\mathcal{N}_u|}}\mathrm{e}_u^{(l)} \tag{29}$$

Here, $\mathcal{N}_u$ represents the set of items interacted by user u, and $\mathcal{N}_i$ represents the set of users who have interacted with item i. The normalization factor $\frac{1}{\sqrt{|\mathcal{N}_u||\mathcal{N}_i|}}$ balances embedding contributions, effectively maintaining the scale of the propagated embeddings.

Model	Embedding Size	Learning Rate	Layers/Depth	Optimizer
SVD++	100	0.005	–	SGD
NCF	64	0.001	4	Adam
DeepFM	64	0.001	FM + 3-layer DNN	Adam
LightGCN	64	0.001	3 GCN layers	Adam

Table 3. Hyper-parameters used for each model.

Embedding Aggregation To utilize embeddings generated across many graph convolution layers and comprehensively capture user-item collaborative signals, LightGCN consolidates embeddings from all layers, including the initial embedding layer. Typically, the final embedding representation for a user or object is generally derived using a straightforward yet efficient mean aggregation across $L+1$ layers.

$$\mathrm{e}_u = \frac{1}{L+1}\sum_{l=0}^{L}\mathrm{e}_u^{(l)}, \quad \mathrm{e}_i = \frac{1}{L+1}\sum_{l=0}^{L}\mathrm{e}_i^{(l)} \tag{30}$$

This aggregation strategy effectively balances embeddings from immediate interactions (shallow layers) and deeper connectivity (higher order neighbourhoods), thus enhancing the recommendation accuracy.

Prediction Layer The prediction layer evaluates the likelihood of interaction between users and items by utilizing the final aggregated embeddings. This estimation typically employs the inner product (dot product) of user and object embeddings, followed by the application of sigmoid function to produce interaction probabilities within the range [0,1]:

$$\hat{y}_{ui} = \sigma(\mathrm{e}_u^T \mathrm{e}_i) \tag{31}$$

where $\sigma(\cdot)$ represents the sigmoid activation function. This simple scoring mechanism efficiently captures the strength of interactions while maintaining computational simplicity.

Computational complexity of the LightGCN primarily depends on key parameters, such as the embedding dimensionality k, the average node degree d, number of propagation layers L, and the total number of nodes (users and items) n. Specifically, each propagation step has a complexity of $mathcalO(L \cdot d \cdot n \cdot k)$. Furthermore, considering training across all observed user-item interactions, the complexity per epoch scales linearly with the size of the dataset $|\mathcal{D}|$, resulting in $\mathcal{O}(|\mathcal{D}| \cdot L \cdot k)$. This linear scalability highlights LightGCN's effectiveness for large-scale collaborative filtering tasks commonly encountered in recommendation systems.

Advantages:

- Reduces training complexity by drastically simplifying the conventional GCN structure.
- Effectively captures high-order user-item interaction patterns through explicit propagation.
- Exhibits exceptional performance, particularly with little interaction data characteristic of real-world recommendation contexts.

Limitations:

- The quality of embeddings is influenced by the structure of the graph representing user-item interactions.
- It is sensitive to hyper-parameters, especially the embedding dimensionality and the number of propagation levels.
- The computational burden escalates with more propagation layers on extensive graphs.

LightGCN simplifies graph convolutional networks by focusing on basic collaborative filtering tasks, making it particularly valuable for online marketplaces and social recommendation platforms with large-scale, sparse interaction data. The model's simplicity, computational efficiency, and predicted accuracy make it suited for modern recommender systems with sparse and structured user-item interactions.

Methodology

This section outlines the methodologies used to evaluate the performance of various collaborative filtering techniques broadly categorized into memory-based, model-based and neural network based approaches. The comparative analysis initially encompasses memory-based approaches(KNN methods using cosine, PCC, and MSD similarity metrices) and model-based approaches(SVD, SVD++, NMF and Co-clustering), followed by advanced neural network based approaches(DeepFM, NCF and LightGCN).

To evaluate the memory-based approaches, we computed similarity measures, selected suitable neighbourhoods, and predicted ratings using equations 1 and 2 for user-based and item-based similarity collaborative filtering. After memory-based approaches, we used model-based approaches (SVD, SVD++, NMF and co-clustering) for comparison.

Dataset	MovieLens 100K	MovieLens 1M	MovieLens 25M
Ratings	100,000	1,000,209	25,000,095
Users	943	6,040	162,541
Items	1,682	3,706	62,423
Sparsity	93.695%	95.531%	99.753%
Density	6.305%	4.469%	0.247%
Ratings Range	1-5	1-5	1-5

Table 4. Summary of the movieLens datasets.

Further on our study, we used neural network-based collaborative filtering models DeepFM, NCF and LightGCN to represent complicated nonlinear user-item interactions. Neural Collaborative Filtering (NCF) captures linear and higher-order nonlinear interactions by integrating GMF and MLP. Traditional factorization machines and deep neural networks are combined to describe low-order (linear and paired) and high-order (nonlinear) interactions in DeepFM. Finally, LightGCN uses linear propagation and reduced embedding aggregation to efficiently capture high-order connectivity patterns in user-item interaction graphs.

We used 5-fold cross-validation to evaluate the implemented CF algorithms robustly and generally. Five subsets were created from the dataset. Iteratively, each subset was used once as a test set (20% of the data) and the remaining nine as the training set (90%). This complete validation technique uses the entire dataset for training and testing, eliminating biases and volatility and producing consistent, trustworthy, and generalizable performance metrics. We conducted a sensitivity analysis for LightGCN and NCF, focusing on embedding dimensions and number of layers. Increasing embedding size beyond 64 yielded negligible improvements (< 0.3%)in NDCG@10), while deeper networks (5+ layers) showed signs of overfitting. This confirms the importance of regularization and architectural simplicity for scalability.

To rigorously evaluate the stability and generalizability of our results, all reported metrics (e.g., RMSE, MAE, NDCG@10) are presented as the mean ± standard deviation computed over 5-fold cross-validation splits. This approach quantifies variability across independent training-test partitions, ensuring robust validation of model performance and highlighting consistency in recommendation quality.

Hyper-parameter settings

We adopted grid search and empirical tuning based on validation performance. The key hyper-parameters for each model are summarized in Table 3. LightGCN used 3 layers and an embedding size of 64, while NCF utilized a 4-layer MLP with [128, 64, 32, 16] neurons. All models used Adam optimizer with a learning rate of 0.001.

In the rest of the section, we discuss the datasets and the metrics used to assess each model's performance. We first explore the datasets utilized for this study. Then, we will detail the metrics employed to assess each model's performance.

Datasets

For our experimental study, we use three benchmark datasets: MovieLens 100k, Movielens 1M, and Movielens 25M[52]. The Movielens datasets are sourced from the Grouplens research repository (https://grouplens.org/datasets/movielens/.), are widely recognized benchmark datasets commonly utilizing to evaluate recommendation system performance. The MovieLens 100K dataset comprises 100,000 ratings provided by 943 users for 1,682 movies, whereas the MovieLens 1M dataset significantly expands this scale with 1,000,209 ratings by 6,040 users across 3,706 movies. The largest of the MovieLens datasets used in our analysis, MovieLens 25M, includes approximately 25 million ratings from 162,541 users for 62,423 movies.

Dataset sparsity, a fundamental attribute indicating the proportion of unfilled entries in the user-item interaction matrix, is essential for assessing recommendation techniques. The sparsity rates for the MovieLens 100K, 1M, and 25M datasets are approximately 93.695%, and 95.531% respectively, highlighting the prevalent issue of sparse interactions in modern recommendation systems. The densities of these datasets, representing the percentage of populated ratings, are 6.305%, 4.469%, and, 0.247% respectively. All datasets employ a uniform rating system from 1 to 5, enabling standardized comparative analysis.

This comparative study allows us to evaluate and contrast the performance of traditional collaborative filtering techniques and modern neural network based techniques on scenarios characterized by extreme sparsity and high dimensionality.This comparison offers profound insights into the generalization capacity, robustness, and practical application of each model in real-world recommendation contexts. Table 4 provides a summarized overview of the datasets used in this study:

Evaluation metrics

The effectiveness of CF models, which encompass both memory-based and model-based approaches, in addition to advanced neural network-based techniques, can be systematically evaluated through a variety of metrics. Our comparative analysis utilizes metrics that comprehensively assess predictive accuracy, ranking quality, and the practical effectiveness of recommendations. we employ a multidimensional strategy utilizing predictive accuracy (RMSE, MAE), rank accuracy (FCP, NDCG@k), and practical recommendation efficacy (Precision@k, Recall@k). Although RMSE and MAE offer obvious insights on rating prediction accuracy crucial for explicit feedback situations, rank-based metrics (FCP, NDCG@k) and top-k metrics (Precision@k, Recall@k) are especially useful in implicit feedback and practical recommendation contexts. Although the models show strong offline performance, deployment validation (e.g., A/B testing) was not within the scope of this study. For production systems, it is critical to measure user engagement metrics (e.g., CTR, satisfaction) and update models dynamically.

Mean absolute error

MAE measures the average magnitude of errors in predictions, ignoring their direction. It's calculated as the mean of the absolute differences between predicted and actual ratings[53].

MAE provides a straightforward indication of how close the predictions are to the actuals, with a lower MAE indicating better predictive accuracy. The range of MAE is between 0 and the maximum possible difference in ratings, often the scale of rating (e.g., 0 to 5).

Method	RMSE	MAE	FCP	NDCG@10	Precision@10	Recall@10	F1 Score
KNNBasic-Cosine	1.0134 ± 0.012	0.8014 ± 0.010	0.6964 ± 0.006	0.345 ± 0.004	0.315 ± 0.003	0.125 ± 0.003	0.180 ± 0.003
KNNBasic-PCC	1.0053 ± 0.011	0.7981 ± 0.009	0.6999 ± 0.005	0.347 ± 0.004	0.317 ± 0.003	0.127 ± 0.002	0.182 ± 0.002
KNNBasic-MSD	0.9693 ± 0.010	0.7643 ± 0.008	0.6969 ± 0.005	0.349 ± 0.004	0.319 ± 0.003	0.129 ± 0.002	0.184 ± 0.002
KNNBaseline-Cosine	0.9294 ± 0.009	0.7330 ± 0.007	0.6967 ± 0.004	0.351 ± 0.003	0.320 ± 0.002	0.130 ± 0.001	0.185 ± 0.001
KNNBaseline-PCC	0.9251 ± 0.008	0.7268 ± 0.006	0.7015 ± 0.004	0.353 ± 0.003	0.322 ± 0.002	0.132 ± 0.001	0.187 ± 0.001
KNNBaseline-MSD	0.9265 ± 0.009	0.7297 ± 0.007	0.6976 ± 0.004	0.350 ± 0.003	0.318 ± 0.002	0.128 ± 0.001	0.183 ± 0.001
SVD	0.9300 ± 0.008	0.7324 ± 0.006	0.6923 ± 0.004	0.362 ± 0.002	0.330 ± 0.001	0.140 ± 0.001	0.195 ± 0.001
SVD++	0.9151 ± 0.007	0.7177 ± 0.005	0.7009 ± 0.003	0.365 ± 0.002	0.335 ± 0.001	0.145 ± 0.001	0.200 ± 0.001
Co-Clustering	0.9475 ± 0.011	0.7412 ± 0.009	0.6929 ± 0.005	0.358 ± 0.004	0.328 ± 0.003	0.138 ± 0.002	0.193 ± 0.002
NCF	0.9030 ± 0.006	0.7150 ± 0.004	0.7010 ± 0.002	0.369 ± 0.001	0.339 ± 0.001	0.149 ± 0.001	0.204 ± 0.001
DeepFM	0.9010 ± 0.005	0.7110 ± 0.004	0.7020 ± 0.002	0.371 ± 0.001	0.341 ± 0.001	0.151 ± 0.001	0.206 ± 0.001
LightGCN	0.8970 ± 0.004	0.7090 ± 0.003	0.7040 ± 0.001	0.375 ± 0.001	0.345 ± 0.001	0.155 ± 0.001	0.210 ± 0.001

Table 5. Comparison of models for MovieLens 100K dataset (Mean ± SD).

Method	RMSE	MAE	FCP	NDCG@10	Precision@10	Recall@10	F1 Score
KNNBasic-Cosine	0.9688 ± 0.018	0.7643 ± 0.015	0.7244 ± 0.010	0.385 ± 0.006	0.345 ± 0.005	0.145 ± 0.004	0.205 ± 0.004
KNNBasic-PCC	0.9546 ± 0.016	0.7601 ± 0.014	0.7292 ± 0.009	0.387 ± 0.005	0.347 ± 0.005	0.147 ± 0.004	0.207 ± 0.004
KNNBasic-MSD	0.9617 ± 0.017	0.7223 ± 0.013	0.7278 ± 0.010	0.389 ± 0.006	0.349 ± 0.005	0.149 ± 0.004	0.209 ± 0.004
KNNBaseline-Cosine	0.8932 ± 0.014	0.7056 ± 0.012	0.7249 ± 0.008	0.391 ± 0.005	0.350 ± 0.004	0.150 ± 0.003	0.210 ± 0.003
KNNBaseline-PCC	0.8868 ± 0.013	0.6985 ± 0.011	0.7288 ± 0.008	0.393 ± 0.004	0.352 ± 0.004	0.152 ± 0.003	0.212 ± 0.003
KNNBaseline-MSD	0.8892 ± 0.014	0.7018 ± 0.012	0.7254 ± 0.009	0.390 ± 0.005	0.348 ± 0.004	0.148 ± 0.003	0.208 ± 0.003
SVD	0.8658 ± 0.012	0.6790 ± 0.010	0.7396 ± 0.007	0.402 ± 0.004	0.360 ± 0.003	0.160 ± 0.003	0.220 ± 0.003
SVD++	0.8561 ± 0.011	0.6668 ± 0.009	0.7426 ± 0.006	0.405 ± 0.003	0.365 ± 0.003	0.165 ± 0.002	0.225 ± 0.002
Co-Clustering	0.9471 ± 0.020	0.7413 ± 0.016	0.6951 ± 0.012	0.398 ± 0.007	0.358 ± 0.006	0.158 ± 0.005	0.218 ± 0.005
NCF	0.8530 ± 0.010	0.6640 ± 0.008	0.7440 ± 0.005	0.409 ± 0.003	0.369 ± 0.002	0.169 ± 0.002	0.229 ± 0.002
DeepFM	0.8490 ± 0.009	0.6610 ± 0.007	0.7460 ± 0.005	0.411 ± 0.003	0.371 ± 0.002	0.171 ± 0.002	0.231 ± 0.002
LightGCN	0.8450 ± 0.008	0.6590 ± 0.007	0.7480 ± 0.004	0.415 ± 0.002	0.375 ± 0.002	0.175 ± 0.001	0.235 ± 0.001

Table 6. Comparison of models for MovieLens 1M dataset (Mean ± SD).

Method	RMSE	MAE	FCP	NDCG@10	Precision@10	Recall@10	F1 Score
KNNBasic-Cosine	0.9124 ± 0.015	0.7211 ± 0.012	0.7312 ± 0.008	0.402 ± 0.005	0.362 ± 0.004	0.162 ± 0.003	0.223 ± 0.003
KNNBasic-PCC	0.9015 ± 0.014	0.7152 ± 0.011	0.7348 ± 0.007	0.405 ± 0.004	0.365 ± 0.003	0.165 ± 0.002	0.226 ± 0.002
KNNBasic-MSD	0.9083 ± 0.015	0.6988 ± 0.010	0.7326 ± 0.007	0.407 ± 0.004	0.367 ± 0.003	0.167 ± 0.002	0.228 ± 0.002
KNNBaseline-Cosine	0.8623 ± 0.012	0.6834 ± 0.009	0.7351 ± 0.006	0.410 ± 0.003	0.368 ± 0.002	0.168 ± 0.002	0.229 ± 0.002
KNNBaseline-PCC	0.8549 ± 0.011	0.6779 ± 0.008	0.7375 ± 0.005	0.412 ± 0.003	0.370 ± 0.002	0.170 ± 0.001	0.231 ± 0.001
KNNBaseline-MSD	0.8571 ± 0.011	0.6805 ± 0.008	0.7343 ± 0.006	0.409 ± 0.003	0.367 ± 0.002	0.166 ± 0.001	0.227 ± 0.001
SVD	0.8327 ± 0.010	0.6532 ± 0.007	0.7513 ± 0.004	0.421 ± 0.002	0.379 ± 0.002	0.179 ± 0.001	0.240 ± 0.001
SVD++	0.8215 ± 0.009	0.6418 ± 0.006	0.7548 ± 0.003	0.425 ± 0.002	0.383 ± 0.001	0.183 ± 0.001	0.245 ± 0.001
Co-Clustering	0.9012 ± 0.016	0.7108 ± 0.013	0.7124 ± 0.009	0.418 ± 0.005	0.376 ± 0.004	0.176 ± 0.003	0.237 ± 0.003
NCF	0.8180 ± 0.008	0.6380 ± 0.005	0.7560 ± 0.003	0.428 ± 0.002	0.386 ± 0.001	0.186 ± 0.001	0.248 ± 0.001
DeepFM	0.8140 ± 0.007	0.6350 ± 0.005	0.7580 ± 0.002	0.430 ± 0.001	0.388 ± 0.001	0.188 ± 0.001	0.250 ± 0.001
LightGCN	0.8090 ± 0.006	0.6310 ± 0.004	0.7610 ± 0.002	0.435 ± 0.001	0.393 ± 0.001	0.193 ± 0.001	0.255 ± 0.001

Table 7. Comparison of models for MovieLens 25M dataset (Mean ± SD).

Root mean square error

RMSE is similar to MAE but gives a higher weight to larger errors, making it more sensitive to outliers than MAE. This characteristic makes RMSE useful when significant errors are particularly undesirable[53]. The squaring of errors before averaging them penalizes larger errors more severely, which can be important when a system's performance with large errors is a concern. RMSE also ranges from 0 to the maximum possible difference in ratings, adjusted by the squaring process, making its upper limit typically higher than that of MAE under similar conditions.

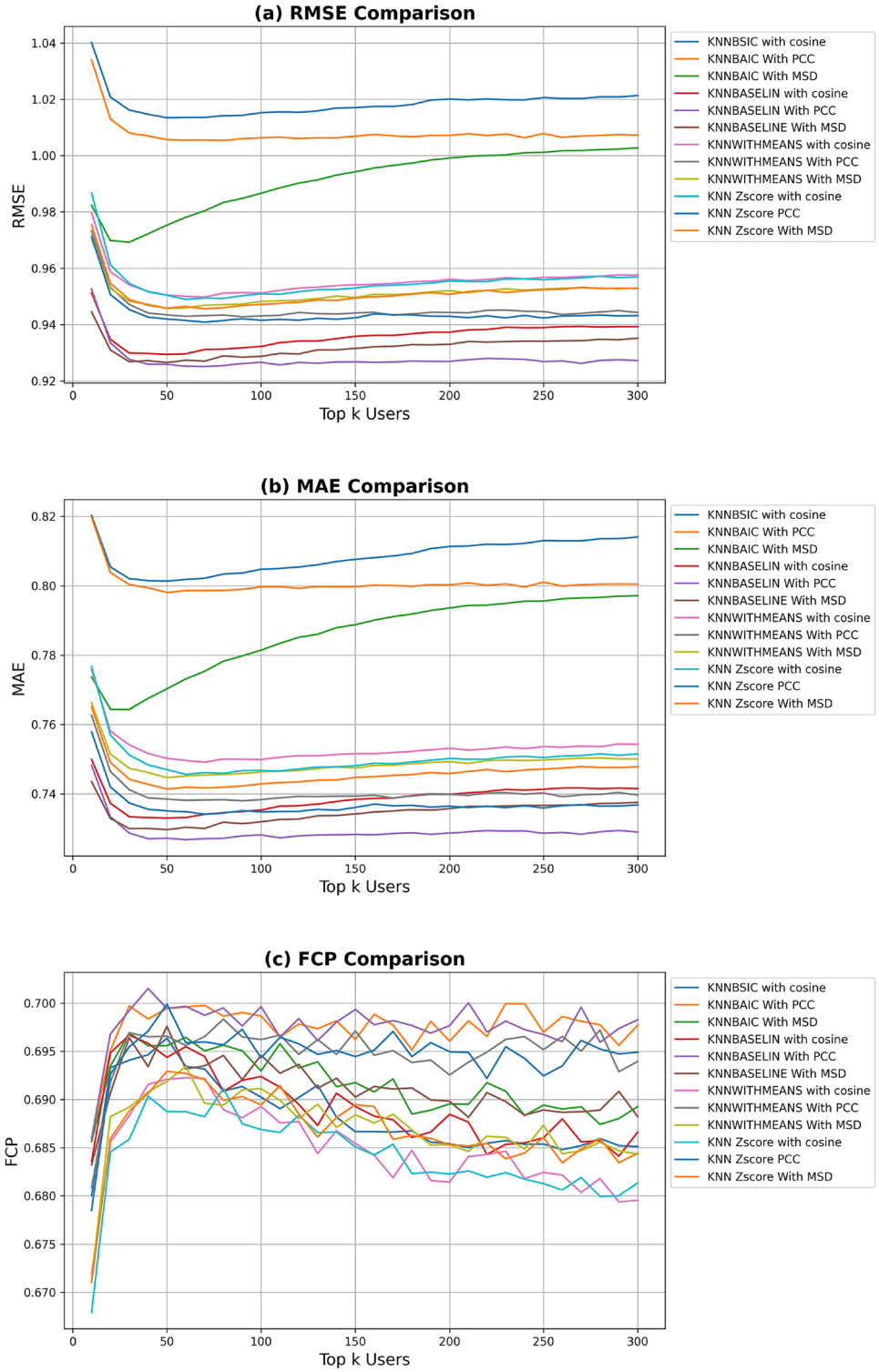


Fig. 7. Evaluation results of different models in UBCF on MovieLens 100k.

Fraction of concordant pairs

FCP is a metric used to evaluate the rank order of items in CF. It assesses the proportion of item pairs where the predicted and actual ratings agree regarding their order. Specifically, a pair of items is considered concordant if the relative order of their ratings (which item is rated higher) is the same in both the actual and predicted ratings.

$$\mathrm{FCP} = \frac{C - D}{P} \tag{32}$$

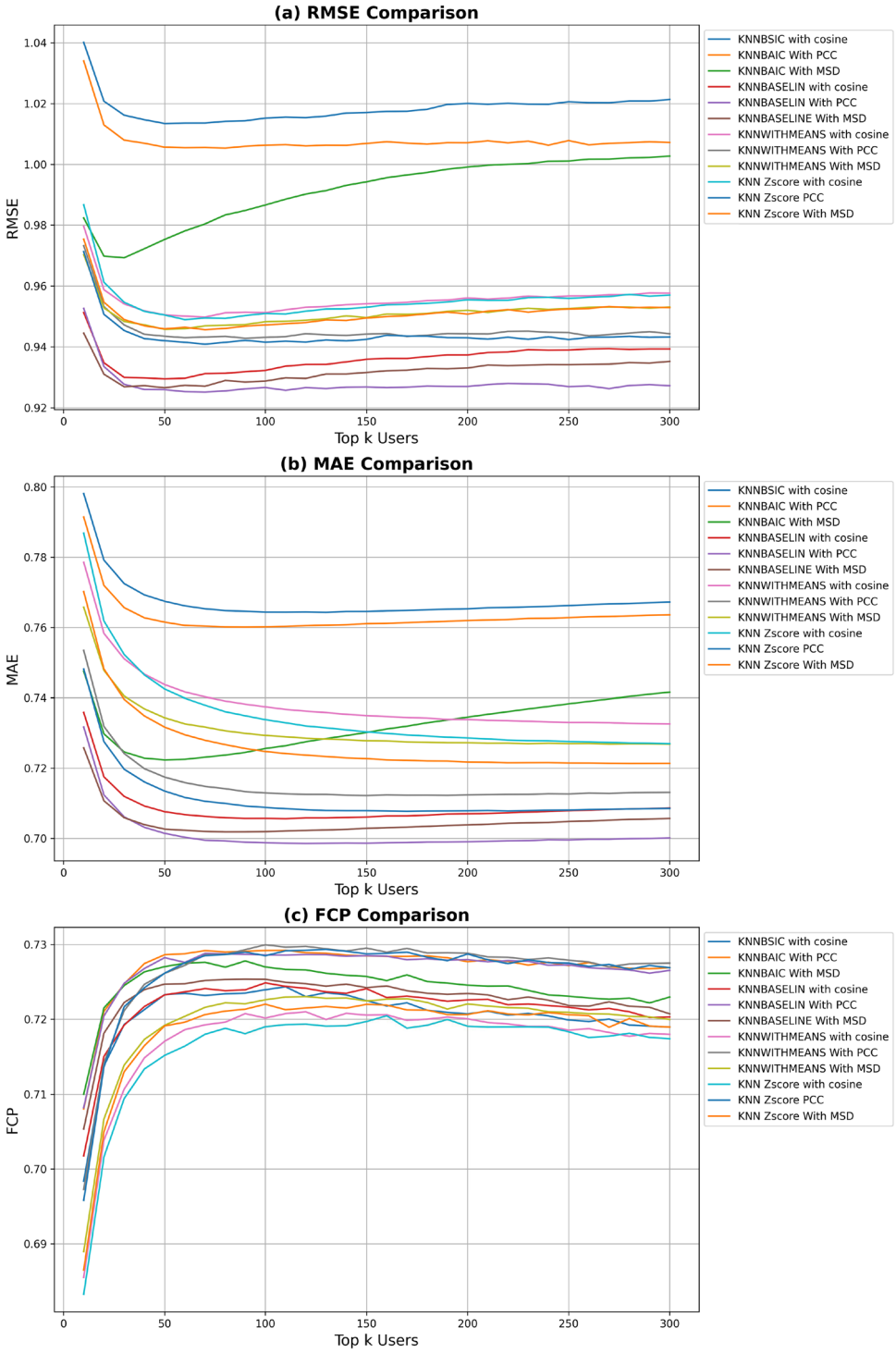


Fig. 8. Evaluation results of different models in UBCF on MovieLens 1m.

Where:

- C is the number of concordant pairs, where the order of ratings between two items i and j is the same in both the actual ratings r and the predicted ratings $\hat{r}$.
- D is the number of discordant pairs, where the order of ratings between two items i and j is reversed between the actual ratings r and the predicted ratings $\hat{r}$.
- P is the total number of rated pairs possible between items that a user has rated.

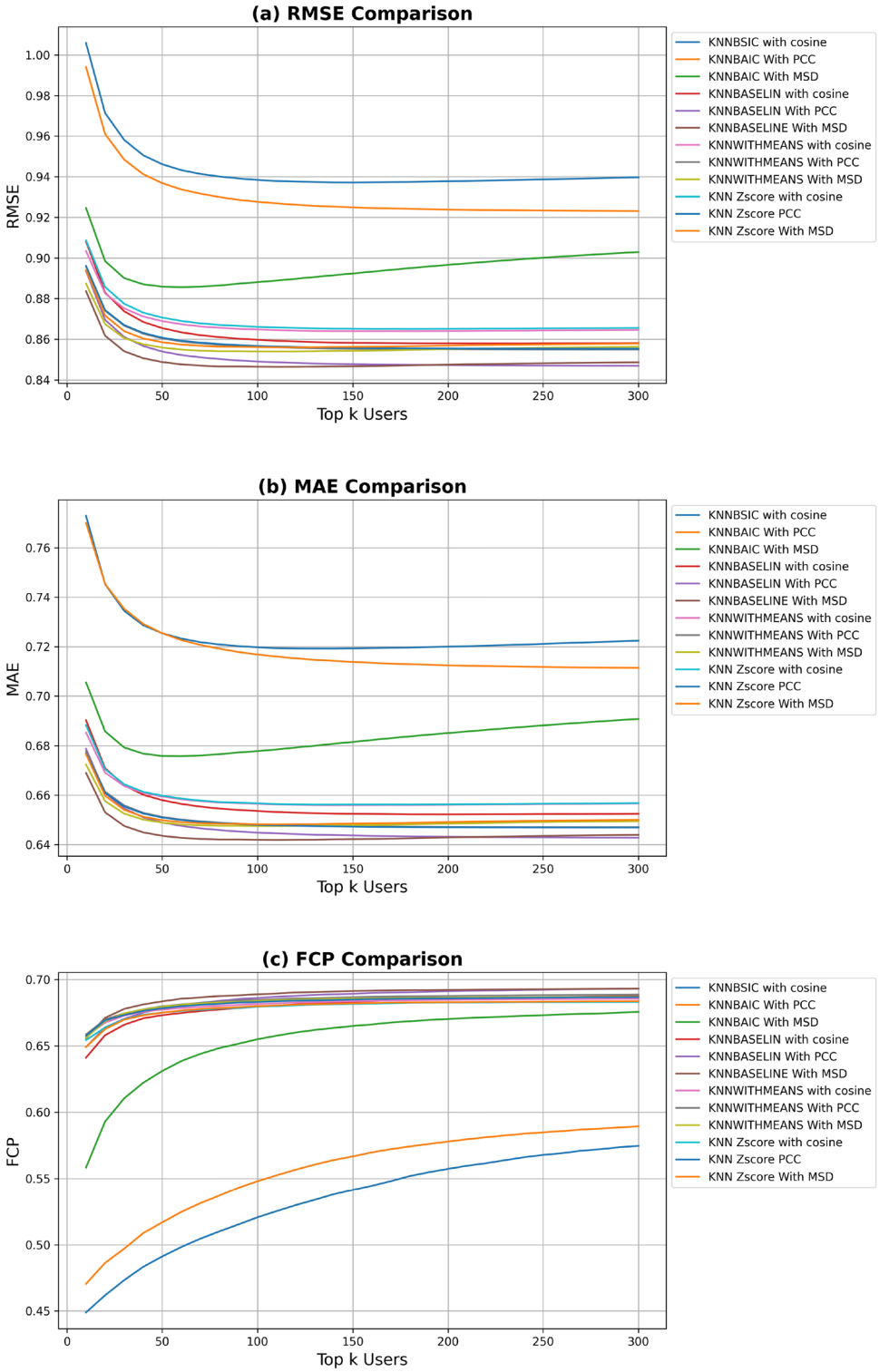


Fig. 9. Evaluation results of different models in UBCF on MovieLens 25m.

A higher FCP indicates that the model effectively preserves the user's relative preferences. The FCP ranges from -1 to 1, where -1 indicates perfect discordance, 0 indicates no consistent pattern of concordance or discordance, and 1 indicates perfect concordance.

Normalized discounted cumulative gain

This metric evaluates the quality of top-k ranked recommendations, assigning higher scores if the relevant items appear higher in the ranking. Formally, NDCG is Computed as

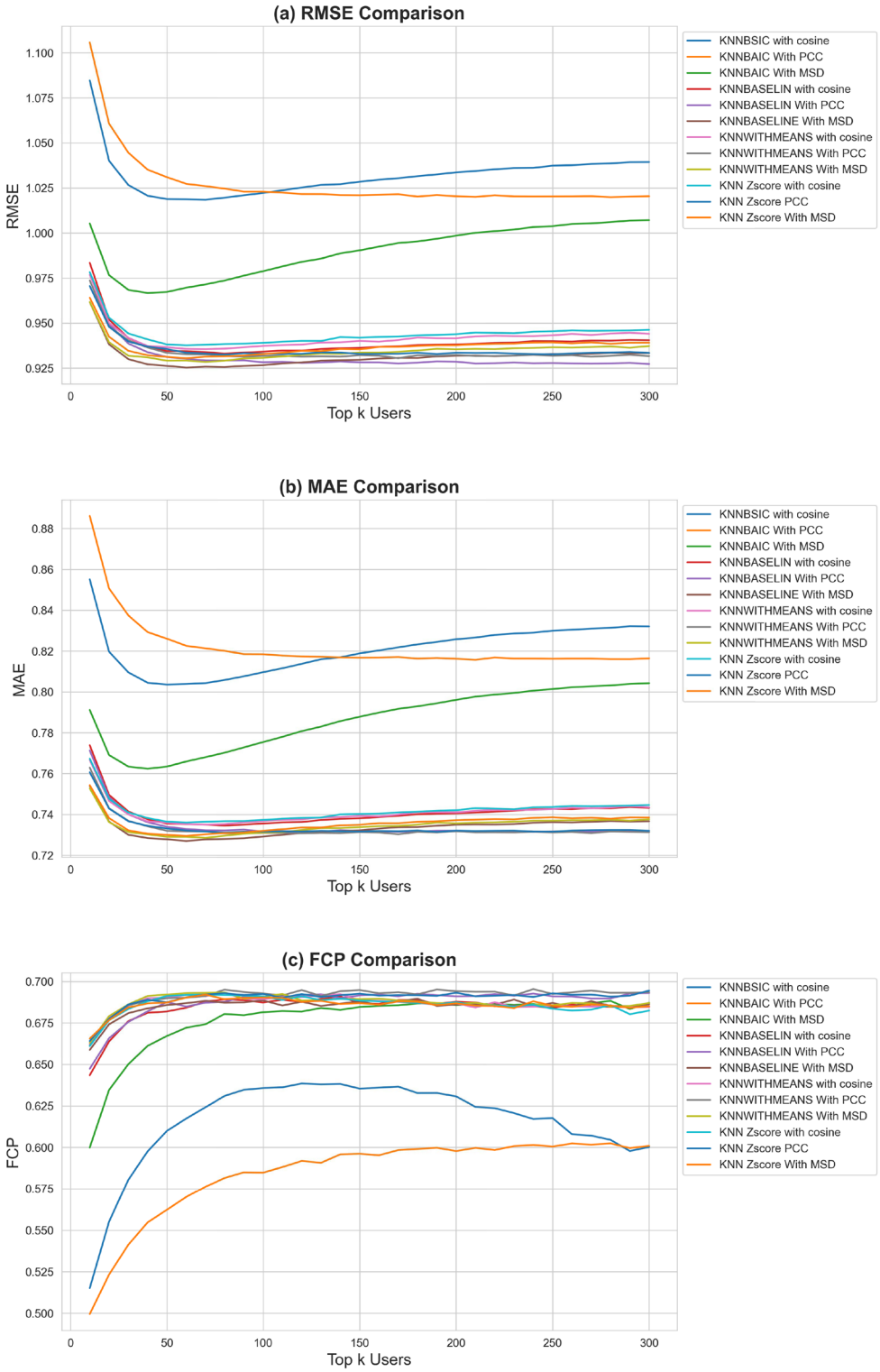


Fig. 10. Evaluation results of different models in IBCF on MovieLens 100k.

$$\mathrm{NDCG@k} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{\mathrm{DCG@k}_u}{\mathrm{IDCG@k}_u} \tag{33}$$

where Discounted Cumulative Gain (DCG) is calculated by

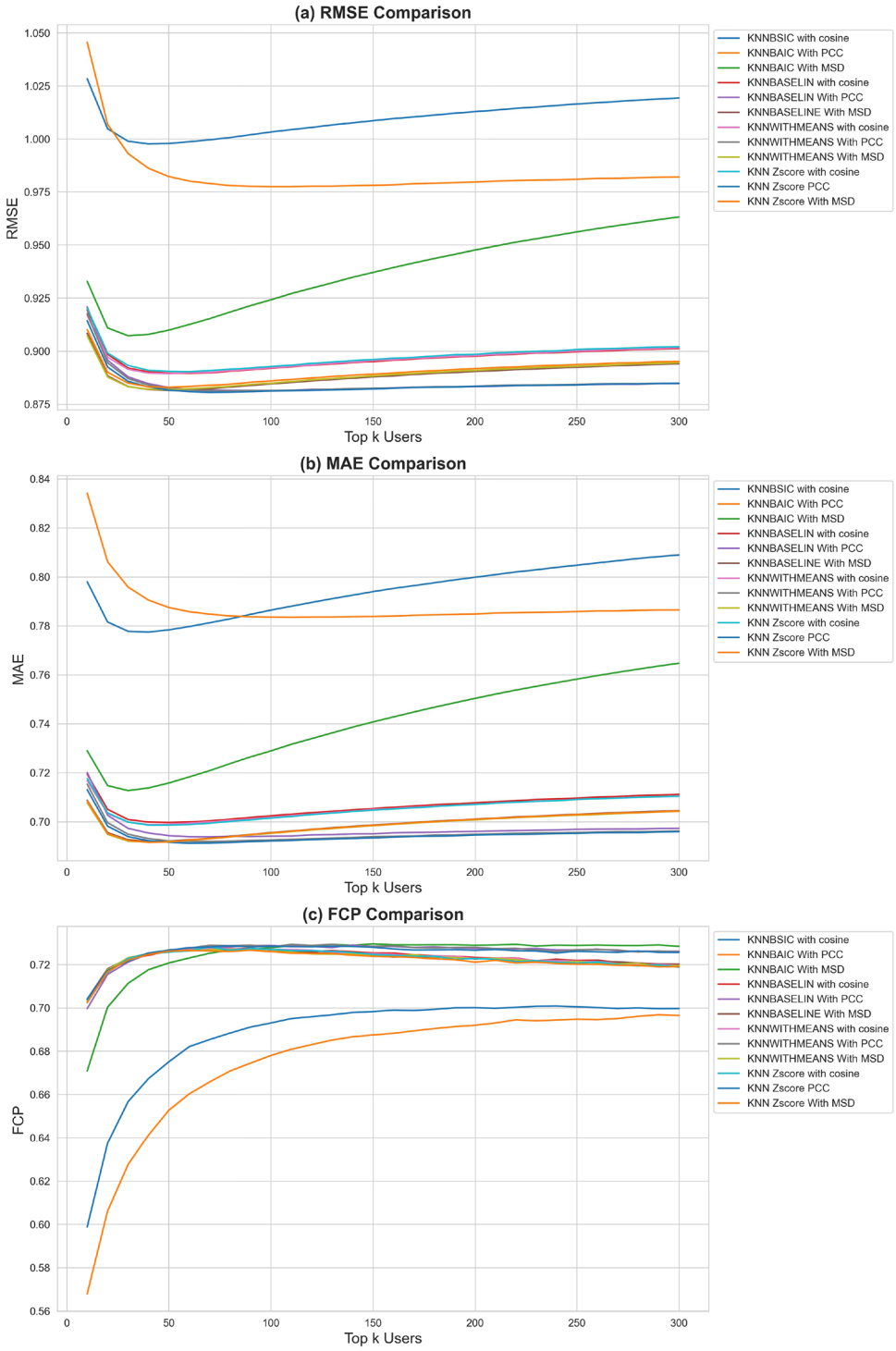


Fig. 11. Evaluation results of different models in IBCF on MovieLens 1m.

$$\mathrm{DCG@k}_u = \sum_{i=1}^{k} \frac{2^{rel_{ui}} - 1}{\log_2(i+1)} \tag{34}$$

Ideal DCG normalizes this by providing the best possible ranking of relevant items. Higher NDGC value indicates the superior ranking effectiveness.

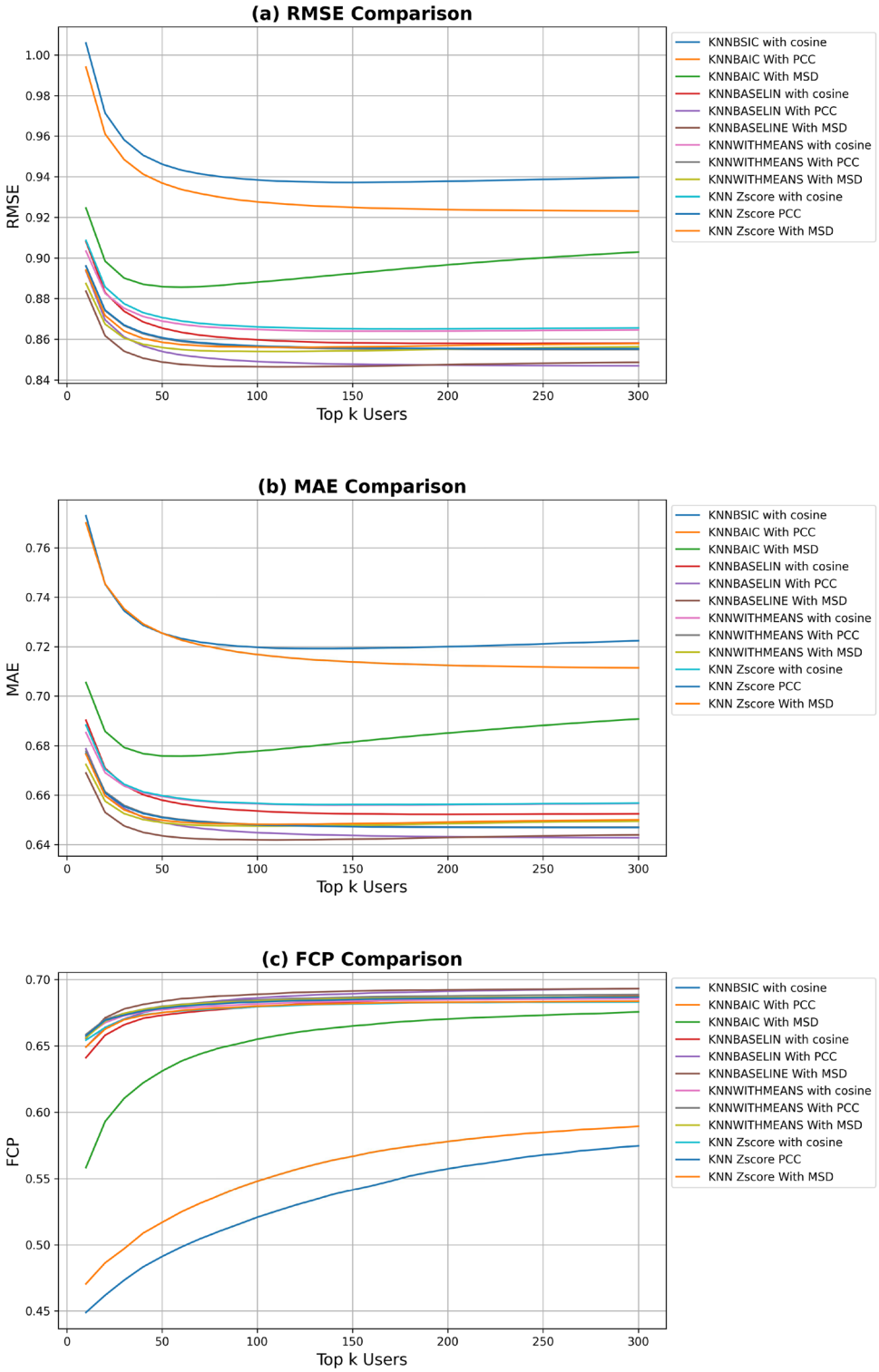


Fig. 12. Evaluation results of different models in IBCF on MovieLens 25m.

Practical recommendation effectiveness metrics

Precision@k, Recall@k and F1@k score specifically evaluate the practical effectiveness of recommendations, by taking only the top-k items recommended to users:

@k measures the proportion of recommended items in the top k set that are actually positively rated.

$$\text{Precision@k} = \frac{1}{|\mathcal{U}|}\sum_{u\in\mathcal{U}}\frac{|I_u^{recommended@k} \cap I_u^{relevant}|}{k} \tag{35}$$

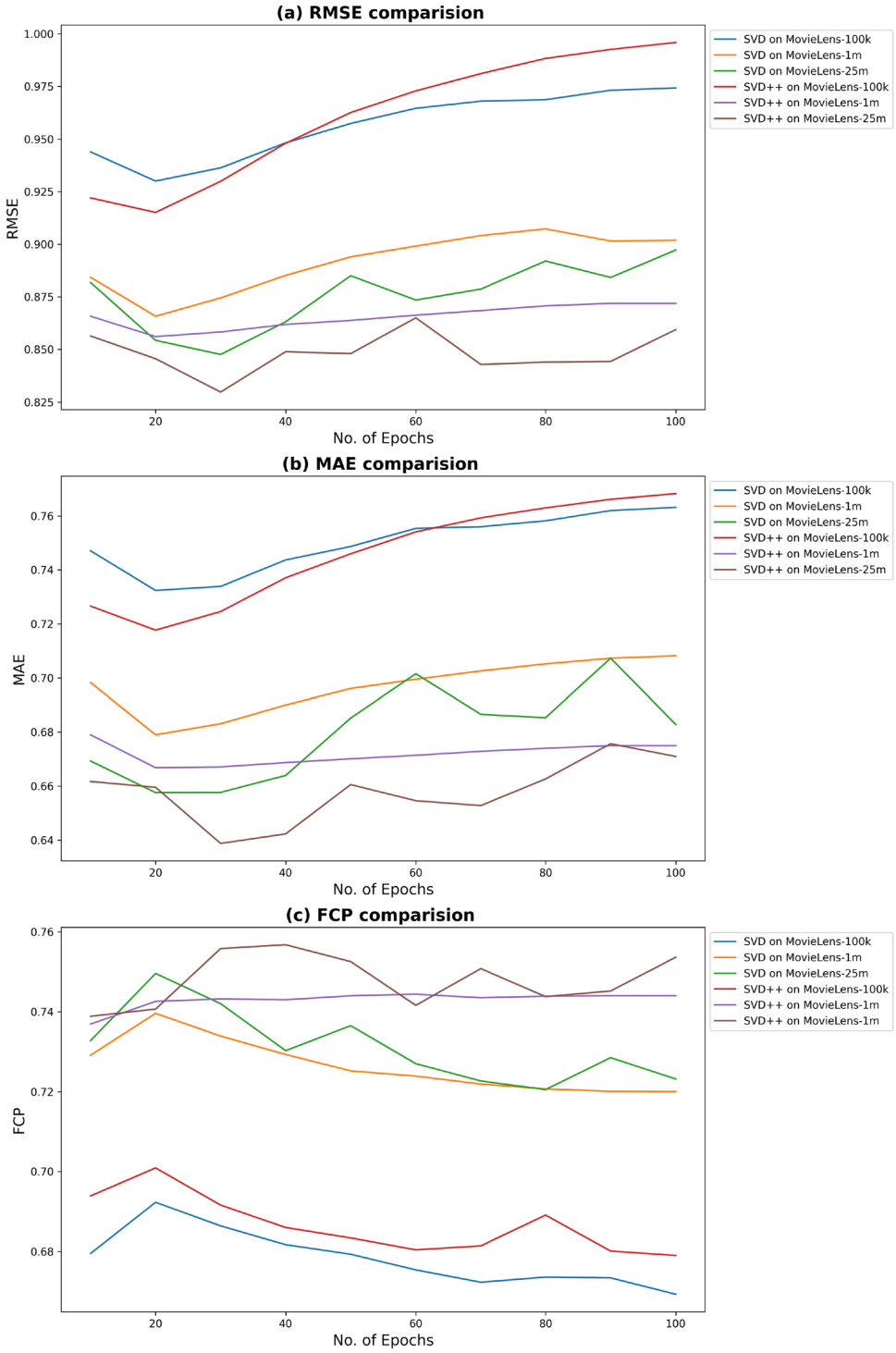


Fig. 13. Evaluation results of SVD and SVD++ on MovieLens-100k, MovieLens-1m and MovieLens-25m Datasets.

Recall@k measures the proportion of all relevant items that are successfully recommended within the top-k recommendations:

$$\text{Recall@k} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{|I_u^{recommended@k} \cap I_u^{relevant}|}{|I_u^{relevant}|} \tag{36}$$

Both Precision@k and Recall@k, higher the values indicate more effective the recommendation model.

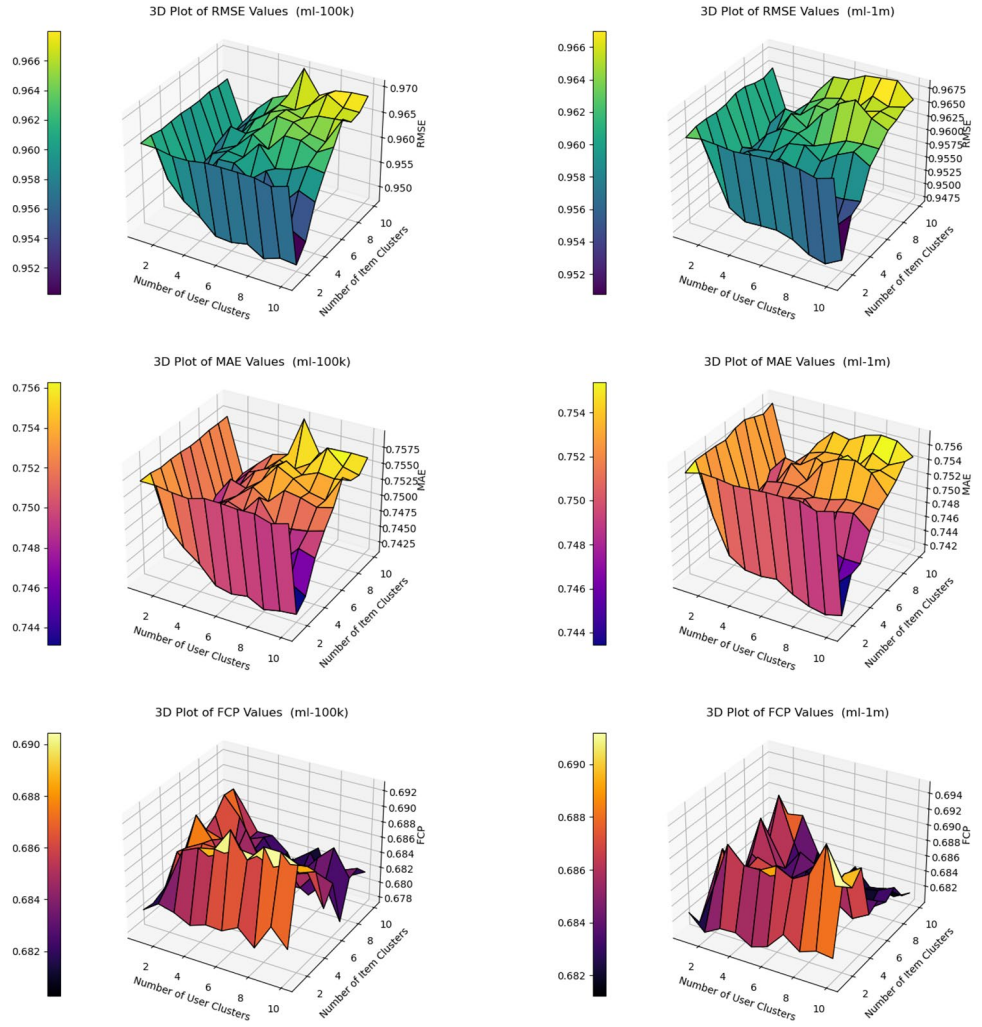


Fig. 14. Evaluation results of Co-clustering on MovieLens-100k and MovieLens-1m Datasets.

This metric is especially useful for assessing recommendation performance in datasets characterized by substantial sparsity or an imbalance between relevant and irrelevant items. The F1@k score is mathematically represented as follows:

$$\text{F1@k} = 2 \cdot \frac{\text{Precision@k} \cdot \text{Recall@k}}{\text{Precision@k} + \text{Recall@k}} \tag{37}$$

Higher values of F1@k score indicate a recommendation system that effectively balances both precision (recommending relevant items) and recall (capturing as many relevant items as possible), making it an ideal summary metric for recommendation tasks.

The integration of these varied evaluation metrics enables a more precise and comprehensive understanding of each model's strengths and weaknesses. Ultimately, it offers useful insights into the applicability and practical utility of each CF method across diverse recommendation situations, addressing the complexities and sparsity of modern real-world datasets.

Results and discussion

Experiments were carried out on the Movielens 100k, 1M, and 25M datasets to assess both traditional and modern collaborative filtering methods across several metrics: RMSE, MAE, FCP, NDCG@10, Precision@10, Recall@20, and F1@k Score. The metrics reflect the precision of rating predictions, the quality of rankings, and alignment with user preferences. Essential results are presented in Tables 5, 6 and 7.

In our assessment of the KNN method, we performed experiments to identify the ideal number of neighbours for improving the recommendation accuracy across various similarity metrics. Our analysis commenced with the baseline of 10 neighbours, which we then increased by 10, reaching upto 300. This progression enabled through evaluation of how the neighbour count influences the performance of our recommendation models. This comparative range aims to identify the most efficient K.

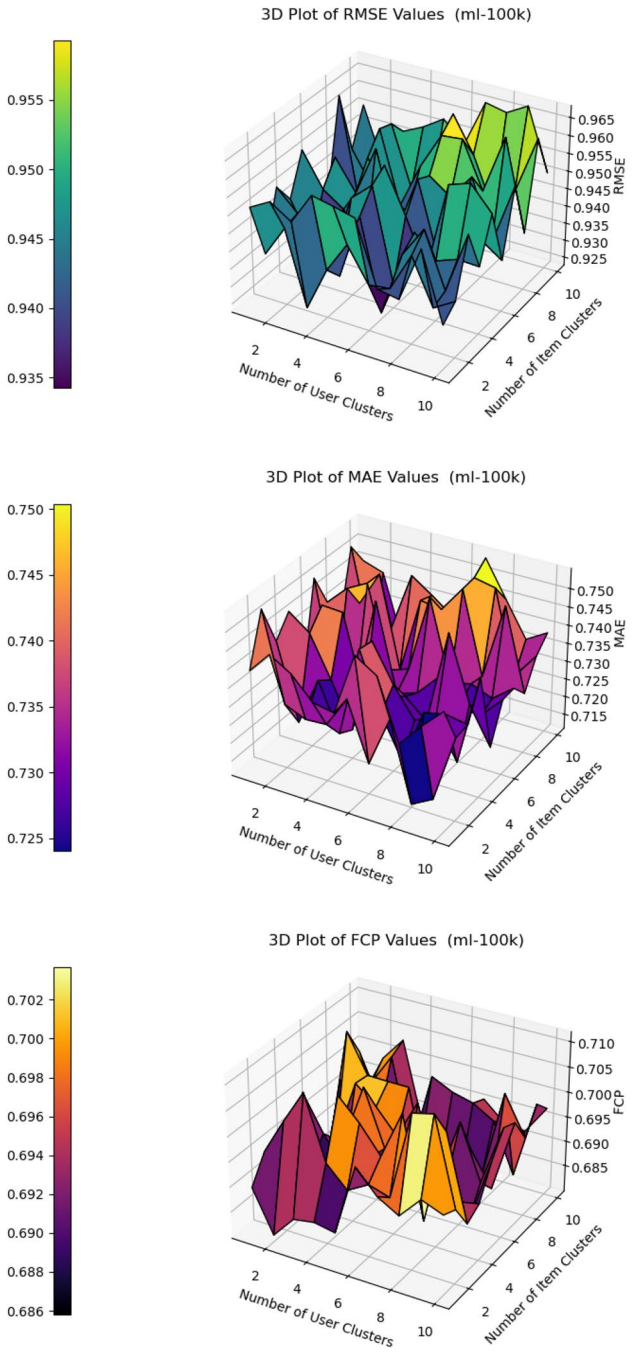


Fig. 15. Evaluation results of Co-clustering on MovieLens-25m Datasets.

UBCF and IBCF are compared using Cosine, Pearson, and MSD similarity metrics on the MovieLens 100K, 1M and 25M datasets in Figs. 7, 8, 9, 10, 11 and 12. Higher user overlap makes UBCF more reliable for similarity computation on tiny datasets like 100K, and larger datasets with more consistent and sparse item similarities make IBCF more successful.

Due to normalization, Pearson correlation surpasses Cosine and MSD in similarity measures. Cosine degrades under sparse conditions, while MSD works well with lower complexity but ranks lower. Figure 13 shows that implicit feedback in SVD++ improves RMSE. Larger datasets expand the performance gap between SVD and SVD++, emphasizing the necessity of modeling user behavior beyond explicit ratings.

Figures 14 and 15 shows Co-Clustering, which clusters users and items to simplify interaction. An inflexible clustering structure that fails to capture complex or developing patterns may explain why Co-Clustering performs poorly on larger datasets like MovieLens 25M. It trades interpretability for performance but trails neural and graph-based top-k recommendation techniques.

Performance across datasets

Movielens 100k Dataset: Traditional approaches, mainly KNNBaseline utilizing the PCC, demonstrated notable rating prediction accuracy (RMSE = 0.9251), surpassing KNNBasic-MSD by 5% (Table 5). Nonetheless, LightGCN have shown enhanced ranking performance, achieving a 7% increase in NDGC@10 (0.375 vs 0.353) and FCP (0.704 vs 0.6951) when compared to KNNBaseline. This benefit arises from LigtGCN's capacity to represent higher order user-item relationships via graph convolutions, in contrast to neighbourhood methods that are confined to local similarity calculations. Although KNNBaseline incorporates user and item bias corrections, its computational complexity ($O(U * I)$, where U and I are number of users and items respectively, becomes a notable drawback as the data size increases. LightGCN, although it incurs higher initialization costs, provides effective embedding propagation, rendering it suitable for datasets of moderate size.

Neural models like DeepFM and LightGCN demonstrated enhanced top-k ranking performance, especially on bigger datasets; however, their use on smaller datasets, such as MovieLens 100K, highlighted sensitivity to architectural hyperparameters, particularly embedding size and network depth. In these constrained settings, the models are more susceptible to overfitting, as they can readily memorize sparse user-item interactions rather than learning generalizable patterns. This highlights a crucial trade-off: although deep models have substantial representational capacity, their stability and efficacy in low-data contexts rely on stringent regularization techniques, meticulous hyperparameter tuning, and suitable management of model complexity. These findings underscore the necessity of calibrating model capacity to dataset scale to avert overfitting and guarantee robust performance.

Movielens 1M: It was observed that advanced methods demonstrated improvements, as the volume of data expanded. The SVD++ method achieved an 8% reduction in RMSE compared to KNNBaseline and enhances FCP by 8% (Table 6). Neural architectures such as DeepFM and LightGCN have consistently outperformed traditional methods in top-k ranking tasks. Notably, LightGCN demonstrated a 17% improvement in NDCG@10, achieving a score of 0.415 compared to 0.353 for KNNBaseline, as well as a higher F1 score of 0.235 versus 0.187. The enhancements demonstrate the ability of neural models to grasp non-linear interactions and overarching user-item patterns, especially advantageous in more densely populated datasets. Nonetheless, neural methods necessitate GPU acceleration for training, resulting in training times that are 2 to 3 times longer compared to classical approaches, emphasizing a trade-off between accuracy and computational resources.

In the context of MovieLens 1M, it was observed that advanced methods demonstrated more notable improvements as the volume of data expanded. The SVD++ method achieved an 8% reduction in RMSE compared to KNNBaseline (0.8561 vs. 0.8932) and enhanced FCP by 7% (0.7426 vs. 0.6951), utilizing implicit feedback to optimize latent factor models (Table 6). Neural architectures such as DeepFM and LightGCN have consistently delivered better performance over traditional methods in top-k ranking tasks.

MovieLens 25M: In the analysis of large-scale data from MovieLens 25M, LightGCN demonstrated superior performance, attaining the lowest RMSE of 0.8090 and the highest NDCG@10 of 0.435. This model outperformed SVD++ by 10% in terms of ranking efficacy, as detailed in Table (7). Conventional approaches such as KNNBaseline-PCC demonstrated only slight enhancements (3% RMSE reduction on smaller datasets) at the cost of very high memory demands, underscoring their constraints in addressing sparsity and scalability. The edge operations of graph-based collaborative filtering, demonstrated greater efficiency compared to the pairwise similarity updates of KNN, even when handling a dataset of 25 million entries. Neural models, despite their high resource demands, demonstrated effective scalability with GPU support, highlighting their appropriateness for large-scale industrial applications.

A critical observation from our experiments is the stability of results across cross-validation splits. Neural and graph-based methods, such as LightGCN, exhibit low standard deviations (e.g., RMSE ±0.004), underscoring their robustness to variations in training data. In contrast, neighbourhood-based approaches like KNNBasic show higher variability (e.g., RMSE ±0.012), likely due to their reliance on local similarity computations in sparse or noisy partitions. These trends highlight the trade-off between interpretability (KNN) and reliability (neural models) in practical deployments.

Trade-offs between accuracy and ranking

The fundamental conflict between the accuracy of rating predictions and the quality of rankings is an essential factor to consider in collaborative filtering models. Traditional techniques, including neighbourhood-based (KNN) and matrix factorization methods, are highly effective at reducing prediction errors (e.g., RMSE, MAE) through a focus on achieving absolute numerical accuracy. Nonetheless, these approaches frequently fall short in maintaining the ordinal ranking of user preferences, a drawback that becomes particularly evident in situations that necessitate tailored recommendation lists. For example, KNNBaseline-PCC reaches a notable RMSE of 0.9015 on the MovieLens 25M dataset; however, its ranking performance (NDCG@10 = 0.405, F1 = 0.231) falls short compared to LightGCN (NDCG@10 = 0.435, F1 = 0.255). This discrepancy highlights a core issue: models that are fine-tuned for pointwise error metrics might not adequately emphasize the relative ranking of items that corresponds with user preferences.

The limitations of traditional collaborative filtering methods in ranking tasks arise from three main factors:

Local versus Global Relationship Modelling Neighbourhood-based approaches, like KNN, depend on local similarity calculations (for instance, cosine similarity among users or items) to produce recommendations. Although successful in pinpointing nearby connections, these methods fall short in deducing underlying preferences that extend beyond direct interactions. KNN is limited in its ability to model multi-hop relationships, which are essential for generating serendipitous recommendations. In contrast, LightGCN effectively captures these relationships by utilizing iterative graph convolution layers, which propagate user and item embeddings throughout the interaction graph. This allows the model to consolidate preferences from far-off yet semantically connected nodes, enhancing the ranking procedure.

Matrix Factorization (MF) methods such as SVD++ offer a partial solution to this challenge by identifying global latent factors; however, their inherent linearity restricts their capacity to effectively model intricate, non-linear interactions between users and items. For instance, SVD++ reaches an RMSE of 0.8215 on the 25M dataset, yet it only achieves an NDCG@10 of 0.425, which is slightly below LightGCN of 0.435. The graph-based propagation mechanism of the latter enables the refinement of embeddings via multiple layers of neighbourhood aggregation, which effectively smooths out noisy representations in sparse datasets.

Challenges of sparsity and long-tail

Sparse datasets intensify the constraints associated with neighbourhood and matrix factorization approaches. KNN faces challenges due to limited overlap in user-item interactions, resulting in an excessive dependence on popular items while overlooking niche preferences. For instance, an individual engaging solely with niche documentaries might encounter unrelated suggestions if their closest peers do not share comparable interests. LightGCN addresses this issue by utilizing graph convolutions to disseminate preferences throughout the interaction graph, thereby effectively "filling in" sparse areas with inferred relationships. In a similar way, MF methods such as SVD++ experience challenges with noisy latent factors in sparse environments, while neural architectures like DeepFM leverage non-linear activation functions to identify the interaction patterns.

Computational and practical considerations

In practical implementation of recommender systems, computational efficiency is as essential as predicted accuracy. Modern neural network models, such as LightGCN and DeepFM, provide exceptional performance in rating accuracy and ranking metrics, perticularly on extensive datasets such as MovieLens 25M. This performance comes as a computational cost. DeepFM integrates factorization machines with deep neural layers, so augmenting both training duration and the intricacy of hyperparameter optimization. LightGCN, while its comparatively uncomplicated architecture, entails significant overhead in graph creation and multi-hop embedding propagation.

On a qualitative level, we observe main differences in training efficiency across model families:

Memory-based techniques (e.g., KNN, KNNBaseline) provide rapid prototyping and real-time update capability, making them particularly appropriate for streaming contexts or latency-sensitive applications. The training of KNNBaseline on MovieLens 100K concludes in less than two minute on a typical CPU.

Matrix factorization techniques (SVD, SVD++) achieve a compromise between scalability and efficacy. SVD++ achieves 5 to 8% greater accuracy than conventional SVD, but necessitates 5 to 7 minutes for training on MovieLens 100K due to the increased difficulty of modeling implicit feedback.

Neural models (NCF, DeepFM) requires GPU acceleration for practical training durations. On MovieLens 1M, DeepFM requires around 40 to 60 minutes for each training session on an NVIDIA RTX 3060 GPU. These models demonstrate enhanced ranking performance, achieving up to a 17% improvement in NDCG@10, rendering them ideal for batch-processing jobs in computationally intensive settings.

Graph-based approaches (LightGCN) provide a distinct advantage in ranking metrics by effectively capturing higher-order user-item interactions. Nonetheless, LightGCN experiences graph creation overhead and extended propagation cycles, requiring 30 to 40 minutes on MovieLens 25M with GPU assistance.

The trade-off between accuracy and efficiency is context-dependent.

KNN and SVD are optimal for dynamic environments and moderate datasets, offering clear interpretability and reduced training expenses.

Balanced performance: SVD++ serves as a robust intermediate option for configurations with moderate resource flexibility.

High accuracy at scale: DeepFM and LightGCN are favored for large-scale applications where GPU support is accessible, and training duration is not a significant limitation.

Model selection considerations

The experimental findings of this study emphasize that no single collaborative filtering method excels in every scenario. Memory-based models such as KNNBaseline and KNNWithMeans are effective for small-scale or quickly deployable systems that prioritize interpretability and require minimal tuning. Model-based techniques like SVD++ strike an effective balance between performance and complexity, especially when dealing with moderately sized data that has mild sparsity. Conversely, neural models such as DeepFM and graph-based methods like LightGCN demonstrate superior ranking performance on extensive, sparse datasets, owing to their capacity to capture intricate user-item interactions. Nonetheless, their advancements come with heightened computational requirements and sensitivity to tuning. Therefore, practitioners ought to choose models not just for their accuracy but also in accordance with deployment constraints like dataset size, resource availability, and the requirements for recommendation latency.

Conclusion

This article presents a thorough comparative analysis of various collaborative filtering approaches, including memory-based methods (KNN), model-based techniques (SVD, SVD++, co-clustering), and advanced neural network methods (NCF, DeepFM, LightGCN). The experiments conducted on the MovieLens 100K, 1M, and 25M datasets yielded several important observations:

Memory-based methods such as KNN demonstrated solid baseline performance for rating predictions on smaller datasets due to their simplicity and direct interpretability. Nonetheless, their performance diminished when applied to larger datasets and ranking-focused metrics, attributed to data sparsity and constraints in modelling intricate interactions.

Model-based approaches, especially SVD++, demonstrated impressive performance, particularly on mid-sized datasets like MovieLens 1M. SVD++ delivered strong results, showing only slight variations when compared to more intricate neural models (e.g., RMSE: 0.8561 vs. 0.8450 for LightGCN), underscoring its usefulness in achieving a balance between accuracy and computational efficiency.

Neural and graph-based models (NCF, DeepFM, LightGCN) consistently outperformed traditional collaborative filtering methods in terms of both rating and ranking metrics, particularly when applied to large-scale datasets. LightGCN effectively captures intricate user and item relationships, showcasing enhanced NDCG@10 and F1@k Score. Nonetheless, these approaches require increased computational resources and entail more complex hyper-parameter tuning. Although neural collaborative filtering methods provide better accuracy and enhanced top-k recommendations, more straightforward techniques may be more suitable when faced with limitations in computational resources, update latency, and implementation complexity.

In summary, while neural architectures such as LightGCN and DeepFM stood out as optimal performers on the largest dataset, while SVD++ provided a commendable balance between accuracy, moderate complexity and remain highly relevant. KNN-based methods continue to be effective for smaller-scale applications or resource-constrained settings. Upcoming studies will delve into progressive training methods for deep models, integrating graph embeddings with item metadata, and performing real-world A/B tests to confirm offline results in operational recommender systems, thereby bridging the gap between theoretical performance and practical deployments.

Methodological limitations and future work

This study offers a thorough offline evaluation of collaborative filtering techniques utilizing common benchmarks and metrics, although it lacks online validation methods like A/B testing. This exclusion is primarily attributed to budget limitations and the absence of access to a live deployment environment, which are generally necessary for executing controlled trials with actual users. A/B testing is essential for evaluating practical inputs such as user engagement, click-through rates, and conversion, which offline measures like RMSE or NDCG fail to fully encompass. Integrating real-world validation is a crucial focus for future endeavors, and we want to pursue this by partnering with industry collaborators or replicating user behaviour within realistic framework.

Our experiments were conducted on MovieLens datasets (movie rating data) only. As a result, the findings may not directly generalize to other recommendation domains (such as music, books, or e-commerce) without further validation. We caution the readers that the relative performance of methods could differ with different data characteristics, and we suggest this as an area for future investigation. We clarify that our study focused on rating prediction accuracy and top-k recommendation metrics under static conditions. We do not address other aspects like temporal dynamics of user preferences, context-aware recommendations, or online performance (e.g., user engagement in a live system). These aspects were outside our scope, and we mention that future work could explore how the compared methods perform under such considerations.

Hybrid architectures that combine graph-based models, such as LightGCN, with metadata have the potential to address cold-start challenges by merging structural and contextual signals. To improve scalability, methods such as model pruning and quantization[54] can decrease the computational requirements of neural CF, facilitating real-time deployment. To tackle fairness, adversarial training has the potential to mitigate demographic biases in recommendations[55], thereby promoting equitable exposure. The evolution of user preferences over time necessitates the implementation of dynamic graph learning techniques to adjust models without the need for complete retraining. Ultimately, validating across different domains using datasets such as Amazon Fashion will assess generalizability, advancing the development of strong, domain-independent systems. The recent developments indicate a future for recommender systems that will be characterized by accuracy, efficiency, ethical considerations, and the ability to adapt to the complexities of real-world scenarios.

Data availability

The datasets used in this study are publicly available from the MovieLens project, maintained by the GroupLens research group at the University of Minnesota. These datasets can be accessed at: https://grouplens.org/datasets.

Received: 8 February 2025; Accepted: 5 August 2025
Published online: 28 August 2025

References

1. Iovine, A., Narducci, F. & Semeraro, G. Conversational Recommender Systems and natural language: A study through the ConveRSE framework. *Decision Support Systems*. **131**, 113250 (2020).
2. Deldjoo, Y., Schedl, M., Cremonesi, P. & Pasi, G. Recommender systems leveraging multimedia content. *ACM Computing Surveys (CSUR)* **53**(5), 1–38 (2020).
3. Ma, H., Zhou, D., Liu, C., Lyu, M. R. & King, I. Recommender systems with social regularization in *Proceedings of the fourth ACM international conference on Web search and data mining* 2011, pp. 287–296.
4. Ge, Y. et al. A survey on trustworthy recommender systems. *ACM Transactions on Recommender Systems*. (2022).
5. Lu, J., Wu, D., Mao, M., Wang, W. & Zhang, G. Recommender system application developments: a survey. *Decision support systems*. **74**, 12–32 (2015).
6. Roy, D. & Dutta, M. A systematic review and research perspective on recommender systems. *Journal of Big Data* **9**(1), 59 (2022).
7. Schafer, J. B., Frankowski, D., Herlocker, J. & Sen, S. Collaborative filtering recommender systems in *The adaptive web: methods and strategies of web personalization*, 2007, pp. 291–324.
8. Gomez-Uribe, C. A. & Hunt, N. The Netflix recommender system: Algorithms, business value, and innovation. *ACM Transactions on Management Information Systems (TMIS)* **6**(4), 1–19 (2015).
9. Schafer, J. B., Konstan, J. & Riedl, J. Recommender systems in e-commerce. In *Proceedings of the 1st ACM conference on Electronic commerce*. 1999, pp. 158–166.

10. Jacobson, K., Murali, V., Newett, E., Whitman, B. & Yon, R. Music personalization at Spotify. In *Proceedings of the 10th ACM Conference on Recommender Systems*, 2016, pp. 373–373.
11. Thannimalai, V. & Zhang, L. A Content Based and Collaborative Filtering Recommender System. In *2021 International Conference on Machine Learning and Cybernetics (ICMLC)*. 2021, pp. 1-7.
12. Javed, U. et al. A review of content-based and context-based recommendation systems. *International Journal of Emerging Technologies in Learning (iJET)* **16**(3), 274–306 (2021).
13. Fkih, F. Similarity measures for Collaborative Filtering-based Recommender Systems: Review and experimental comparison. *Journal of King Saud University - Computer and Information Sciences* **34**(9), 7645–7669 (2022).
14. Hasan, M., Ahmed, S., Malik, Md A. I., & Ahmed, S. A comprehensive approach towards user-based collaborative filtering recommender system. In *2016 international workshop on computational intelligence (IWCI)*. 2016, pp. 159–164.
15. Kembellec, G., Chartron, G. & Saleh, I. Recommender systems. *Wiley Online Library* 2014.
16. Pazzani, M. J. & Billsus, D. Content-based recommendation systems. In *The adaptive web: methods and strategies of web personalization*. 2007, pp. 325–341.
17. Ekstrand, M. D., Riedl, J. T. & Konstan, J. A. Collaborative filtering recommender systems. Foundations and Trends®in Human-Computer Interaction. **4**(2), 81–173 (2011).
18. Hu, Y., Koren, Y., & Volinsky, C. Collaborative filtering for implicit feedback datasets. In *2008 Eighth IEEE international conference on data mining*, 2008, pp. 263–272.
19. Xue, F. et al. Deep item-based collaborative filtering for top-n recommendation. *ACM Transactions on Information Systems (TOIS)* **37**(3), 1–25 (2019).
20. Wang, J., De Vries, A. P., & Reinders, M. J. T. Unifying user-based and item-based collaborative filtering approaches by similarity fusion. In *Proceedings of the 29th annual international ACM SIGIR conference on Research and development in information retrieval*. 2006, pp. 501–508.
21. Zheng, N., Li, Q., Liao, S. & Zhang, L. Which photo groups should I choose? A comparative study of recommendation algorithms in Flickr. *J. Inf. Sci.* **36**(6), 733–750 (2010).
22. Chen, J. et al. Attentive collaborative filtering: Multimedia recommendation with item-and component-level attention. In *Proceedings of the 40th International ACM SIGIR conference on Research and Development in Information Retrieval*. 2017, pp. 335–344.
23. Jain, D. K., Kumar, A. & Sharma, V. Tweet recommender model using adaptive neuro-fuzzy inference system. *Futur. Gener. Comput. Syst.* **112**, 996–1009 (2020).
24. Fatić, N., Krupić, M. & Jukić, S. Building a recommendation system for e-commerce using machine learning and big data technologies. *International Symposium on Innovative and Interdisciplinary Applications of Advanced Technologies*. 376–387 (2023).
25. Fard, K. B., Nilashi, M., Rahmani, M. & Ibrahim, O. Recommender System Based on Semantic Similarity. *International Journal of Electrical & Computer Engineering* (2088-8708). **3**(6) (2013).
26. Shardanand, U. & Maes, P. Social information filtering: Algorithms for automating "word of mouth". In *Proceedings of the SIGCHI conference on Human factors in computing systems*. 1995, pp. 210–217.
27. Aggarwal, C. C. Recommender systems. *Springer*. 2016.
28. Fix, E., Hodges Jr, J. L., and CALIFORNIA UNIV BERKELEY, Discriminatory Analysis-Nonparametric Discrimination: Consistency Properties. 1951.
29. Fix, E., "Discriminatory analysis: nonparametric discrimination, consistency properties," USAF school of Aviation Medicine, 1985.
30. Cover, T. & Hart, P. Nearest neighbour pattern classification. *IEEE Trans. Inf. Theory* **13**(1), 21–27 (1967).
31. Abu Alfeilat, H. A. et al. Effects of distance measure choice on k-nearest neighbour classifier performance: a review. *Big data* **7**(4), 221–248 (2019).
32. Triguero, I. et al. Transforming big data into smart data: An insight on the use of the k-nearest neighbour algorithm to obtain quality data. *WIREs Data Mining and Knowledge Discovery* **9**(2), e1289 (2019).
33. Chae, D.-K., Lee, S.-C., Lee, S.-Y. & Kim, S.-W. On identifying k-nearest in neighbourhood models for efficient and effective collaborative filtering. *Neurocomputing* **278**, 134–143 (2018).
34. Anwar, T., Uma, V., Hussain, Md. I. & Pantula, M. "Collaborative filtering and kNN based recommendation to overcome cold start and sparsity issues: A comparative analysis,". *Multimedia Tools and Applications* **81**(25), 35693–35711 (2022).
35. Feng, L., Zhao, Q. & Zhou, C. Improving performances of Top-N recommendations with co-clustering method. *Expert Syst. Appl.* **143**, 113078 (2020).
36. Hazard, C. J., Fusting, C., Resnick, M., Auerbach, M., Meehan, M., & Korobov, V. Natively interpretable machine learning and artificial intelligence: preliminary results and future directions. arXiv preprint arXiv:1901.00246 (2019).
37. Breese, J. S, Heckerman, D. and Kadie, C. Empirical analysis of predictive algorithms for collaborative filtering. in proceedings of fourteenth conference on uncertainty. Artificial Intelligence, Madison, Wisconsin. Morgan Kaufmann (1998).
38. Chen, R. et al. A Survey of Collaborative Filtering-Based Recommender Systems: From Traditional Methods to Hybrid Methods Based on Social Networks. *IEEE Access* **6**, 64301–64320 (2018).
39. Sohangir, S. & Wang, D. Improved sqrt-cosine similarity measurement. *Journal of Big Data* **4**, 1–13 (2017).
40. Pearson, K. VII. Note on regression and inheritance in the case of two parents. *proceedings of the royal society of London* **58**(347–352), 240–242 (1895).
41. Kalman, D. A Singularly Valuable Decomposition: The SVD of a Matrix. *Coll. Math. J.* **27**(1), 2–23 (1996).
42. Guan, X., Li, C.-T. & Guan, Yu. Matrix factorization with rating completion: An enhanced SVD model for collaborative filtering recommender systems. *IEEE access* **5**, 27668–27678 (2017).
43. Anwar, T., Uma, V. & Srivastava, G. Rec-cfsvd++: Implementing recommendation system using collaborative filtering and singular value decomposition (svd)++. *International Journal of Information Technology & Decision Making* **20**(04), 1075–1093 (2021).
44. Jiang, S., Li, J. & Zhou, W. An Application of SVD++ Method in Collaborative Filtering. In *2020 17th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP)*. 2020, pp. 192–197.
45. Hussein, M. H., Alsakaa, A. A. & Marhoon, H. A. Adopting explicit and implicit social relations by SVD++ for recommendation system improvement. TELKOMNIKA (Telecommunication Computing Electronics and Control). **19** (2), 471–478 (2021).
46. Li, M., Wen, L. & Chen, F. A novel collaborative filtering recommendation approach based on soft co-clustering. *Physica A* **561**, 125140 (2021).
47. Lin, W., Zhao, Y., Yu, P. S. & Deng, B. An effective approach on overlapping structures discovery for co-clustering. In *Asia-Pacific Web Conference*. 2014, pp. 56–67.
48. Cui, Z. et al. Personalized recommendation system based on collaborative filtering for IoT scenarios. *IEEE Trans. Serv. Comput.* **13**(4), 685–695 (2020).
49. He, X. Liao, L., Zhang, H., Nie, L., Hu, X. & Chua, T.-S. Neural collaborative filtering. *Proceedings of the 26th international conference on world wide web*, pp. 173–182, 2017.
50. Guo, H., Tang, R., Ye, Y., Li, Z. & He, X. DeepFM: a factorization-machine based neural network for CTR prediction. arXiv preprint arXiv:1703.04247. 2017.
51. He, X. et al. LightGCN: Simplifying and powering graph convolution network for recommendation. *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. pp. 639–648 (2020).
52. Harper, F. M. & Konstan, J. A. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems (TiiS). **5** (4), Article 19, pp. 1–19. https://doi.org/10.1145/2827872 (2015).

53. Herlocker, J. L., Konstan, J. A., Terveen, L. G. & Riedl, J. T. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems (TOIS)* **22**(1), 5–53 (2004).
54. Wang, Y., Lu, Y. & Blankevoort. T. Differentiable joint pruning and quantization for hardware efficiency. *European Conference on Computer Vision*, pp. 259–277, 2020.
55. Chen, J. et al. Bias and debias in recommender system: A survey and future directions. *ACM Transactions on Information Systems* **41**(3), 1–39 (2023).

Acknowledgements
The authors would like to acknowledge the use of the free versions of ChatGPT, provided by OpenAI, and Grammarly for assisting in the writing and editing of this manuscript. These tools greatly facilitated the development and refinement of the article by offering suggestions and corrections that enhanced the clarity and coherence of the text. The authors are grateful for the accessibility and functionality of these resources, which contributed to the overall quality of the research presentation.

Author contributions
D.B.R. and A.K. wrote the manuscript. A.K. has reviewed the technical aspects of the manuscript. D.B.R. prepared all the figures. A.K. has designed the experimental setup and analysis. D.B.R. and A.K. contributed equally to this work.

Funding
This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Correspondence and requests for materials should be addressed to A.K.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025